

GlueLUT: Generalized Lookup Table Arguments over Residue Rings via Auxiliary Fields

Yuanju Wei^{1,2}, Zhelei Zhou^{3,✉}, Xinxuan Zhang^{1,2,✉}, Songyu Wu^{1,2}, Binwu Xiang⁴, Cheng Hong³, and Yi Deng^{5,✉}

¹ State Key Laboratory of Cyberspace Security Defense

Institute of Information Engineering, Chinese Academy of Sciences

² School of Cyber Security, University of Chinese Academy of Sciences

³ Ant Group

⁴ East China Normal University

⁵ Xidian University

{weiyuanju, zhangxinxuan, wusongyu}@iie.ac.cn

{zhouzhelei.zzl, vince.hc}@antgroup.com

bwxiang@sc.ecnu.edu.cn, ydeng.cas@gmail.com

Abstract. Lookup Table (LUT) arguments are a central efficiency primitive in modern SNARKs, and existing high-performance constructions are largely tailored to large fields. Meanwhile, an increasingly important class of applications is natively ring-based, with arithmetic carried out over residue rings $\mathbb{Z}_Q := \mathbb{Z}/Q\mathbb{Z}$. We find that naively extending field-based lookup table techniques to rings faces fundamental obstacles, which can lead to unsoundness, limited applicability, or poor efficiency.

We introduce **GlueLUT**, a general framework for constructing LUT arguments over *arbitrary* residue ring $\mathbb{Z}_Q$ that supports *arbitrary* tables. Our main technical tool is a new primitive called *Cross-Modulus Consistency* (CMC) PIOP, proves that two witnesses defined over coprime moduli share the same underlying integer in the canonical range. Using our CMC PIOP as a glue, we perform the lookups over an auxiliary field $\mathbb{F}_P$ (for a prime $P > Q$) and then certify the consistency between the witness over $\mathbb{Z}_Q$ and the witness over $\mathbb{F}_P$, thereby avoiding the obstacles of constructing LUT arguments directly over rings. We further provide two optimized instantiations, **GlueLUT-v1** for $Q = pq$ and **GlueLUT-v2** for $Q = p^k$, capturing common modulus families in practice. Finally, we implement **GlueLUT-v1** and **GlueLUT-v2** as stand-alone PIOPs and report prototype results that corroborate our theoretical efficiency.

Keywords: Lookup table argument · SNARKs · Rings.

1 Introduction

Lookup Table (LUT) Argument [3,21,46,18,41] is a powerful tool to enhance the efficiency of modern Succinct Non-interactive Arguments of Knowledge (SNARKs) [33,37,24,2]. Informally, a LUT argument allows a prover to convince a verifier

that the elements of a witness vector belong to a predefined table. This capability turns costly arithmetic operations into table membership checks and is now indispensable in various applications, including bit decomposition, range proofs, exponentiation proofs, and general set membership proofs.

Why lookups matter even more for SNARKs over rings. While most deployed SNARKs operate over large prime fields, an increasingly important set of applications is natively *ring-based*, with arithmetic carried out over residue rings $\mathbb{Z}_Q$ (and closely related Galois rings). Examples include verifiable computation for homomorphic encryption, verifiable CPU computations, and efficient fixed-/floating-point arithmetic [22,31,29]. Motivated by these targets, recent work has begun to build SNARKs and polynomial commitments directly over residue rings (or Galois rings) [22,44,31,11,45]. However, despite this progress on *ring-native algebraic proof systems*, we still lack efficient and general-purpose LUT arguments over $\mathbb{Z}_Q$.

A taxonomy of LUT arguments over field, and the ring barrier. Over prime fields, lookup arguments admit a broad spectrum of highly optimized constructions. Hafezi et al. [27] systematize the landscape into several prominent families: *multiset equality* [3,21,48], *LogUp-based* (a.k.a., *log-derivative based*) *methods* [26,18,19,38,8,49,17,6,9], *accumulator-based approaches* [20,10,28], and *subvector extraction* (a.k.a., *matrix-vector*) *techniques* [46,39,47,41,40]. More recently, Bünz et al. [7] proposed a novel approach for building LUT arguments based on *permutation checks*.

The central challenge in LUT arguments over rings is that these highly successful paradigms are tailored to fields and often fundamentally rely on field properties. When naively transplanted to residue rings $\mathbb{Z}_Q$, these techniques frequently fail or suffer significant efficiency degradation.

The core obstacle: multiset encodings lose injectivity over rings. Many LUT arguments over fields reduce to checking equality of two multisets (say, A and B). Over a field, one can encode a multiset A via the root product polynomial

$$\Gamma_A(X) := \prod_{a \in A} (X - a).$$

Over a field, this encoding is injective on multisets⁶: hence $\Gamma_A(X) = \Gamma_B(X)$ implies $A = B$.

However, over residue rings $\mathbb{Z}_Q$ with composite Q , injectivity can fail in a fundamental way. This failure occurs already in two distinct regimes (see Section 3.1): (i) when Q has at least two coprime prime-power factors, the Chinese Remainder Theorem (CRT) identifies $\mathbb{Z}_Q \cong \prod_i \mathbb{Z}_{p_i^{k_i}}$, so equality in $\mathbb{Z}_Q[X]$ is equivalent to simultaneous equality in each component, and distinct multisets can induce the same root-product polynomial by “re-pairing” roots across components; e.g., in $\mathbb{Z}_{15}$ the distinct sets $A = \{1, 2\}$ and $B = \{7, 11\}$ satisfy

⁶ Equivalently, over a splitting field, equality of monic root-product polynomials implies equality of the corresponding root multisets (with multiplicity).

$\Gamma_A(X) = \Gamma_B(X)$ in $\mathbb{Z}_{15}[X]$. (ii) even for prime-power moduli $Q = p^k$, the presence of zero divisors can still create collisions in the root-product encoding; for instance, in $\mathbb{Z}_8$ the distinct sets $A = \{1, 7\}$ and $B = \{3, 5\}$ satisfy $\Gamma_A(X) = \Gamma_B(X)$ in $\mathbb{Z}_8[X]$. Consequently, lookup arguments based on multiset equality (including LogUp-style constructions) can be unsound for general tables defined over $\mathbb{Z}_Q$.

There is nevertheless an important positive result: when all table elements lie in an *exceptional set* (where pairwise differences are units), the product encoding becomes injective. Cascudo et al. [11] leverage this phenomenon to build efficient range proofs over rings in the context of verifiable FHE, by decomposing the range table into subsets that lie in an exceptional set. Their work provides a valuable and elegant ring-native primitive, and its applicability depends on being able to confine table values to structured subsets. We note that, when table elements are distributed over the entire ring $\mathbb{Z}_Q$, their technique cannot be directly applied.

Why the other field families also face barriers over rings. The remaining existing lookup argument techniques over fields also face challenges when applied to rings. Accumulator-based constructions typically rely on pairing-based groups or RSA-type accumulators, in which the table elements need to be encoded with prime numbers. While these structures have not been found over residue rings. Subvector extraction techniques are commonly instantiated via KZG-style commitments [32] (hence pairings), or via memory checking technique whose standard realizations again fall back to multiset equality; both aspects complicate a direct extension to general $\mathbb{Z}_Q$. Permutation-check based LUT arguments are more compatible with ring arithmetic: since sumcheck can be instantiated over commutative rings, the permutation-check approach proposed by Bünz et al. [7] could in principle be adapted to $\mathbb{Z}_Q$. The main obstacle is efficiency: in [7], Bünz et al. mentioned that over fields, the sumcheck protocol admits NTT-based optimizations, yielding $\tilde{O}(n\sqrt{\log n})$ prover’s complexity. Over most composite rings $\mathbb{Z}_Q$, however, such NTT optimizations are unavailable, and the resulting prover’s complexity degrades to $O(n \log n)$. Thus, while permutation-check based approaches provide an important baseline, they incur a noticeable performance penalty when moved from fields to rings.

In summary, there is a clear need for efficient and general-purpose LUT arguments over residue rings $\mathbb{Z}_Q$, and existing field-based techniques either fail or suffer efficiency losses when adapted to this setting.

Our solution: lookups over an auxiliary field with a cross-modulus glue. We propose a framework that treats lookups as a representation-level task rather than a ring-arithmetic task. We execute the LUT argument over an *auxiliary field* $\mathbb{F}_P$ (for a prime $P > Q$), where the full toolbox of field-based LUT arguments is available, and then certify that the witness over $\mathbb{F}_P$ matches the witness over $\mathbb{Z}_Q$ as integers in $[0, Q)$. Our key technique is a new primitive called *Cross-Modulus Consistency* (CMC) PIOP that bridges $\mathbb{F}_P$ and $\mathbb{Z}_Q$ via the CRT reasoning and a compact range proof mechanism. Conceptually, this is not a workaround layered on top of ring SNARKs; it is a principled separation of concerns: we enforce membership in the algebraic domain where it is easiest

(a field), and we enforce representation consistency with the target ring via a dedicated glue proof.

1.1 Our Contributions

In this work, we introduce **GlueLUT**, a suite of efficient lookup table arguments for SNARKs whose main arithmetic is over residue rings $\mathbb{Z}_Q$. Our contributions are fourfold.

1. **A new primitive: cross-modulus consistency PIOP.** We propose a new primitive called *Cross-Modulus Consistency* (CMC) PIOP for *arbitrary coprime moduli* P, Q (with $P > Q$), which proves that two committed witnesses $f : B_\mu \rightarrow \mathbb{Z}_P$ and $g : B_\mu \rightarrow \mathbb{Z}_Q$ represent the same underlying integers in the canonical range (Section 4.1, Theorem 1). Technically, it bridges the two domains via an *implicit CRT bridge* $\mathbb{Z}_{PQ}$ and reduces consistency to a compact range proof over $\mathbb{Z}_{PQ}$.
2. **GlueLUT-g: a general-purpose LUT argument over rings.** We present GlueLUT-g, a general framework that composes (i) *any* off-the-shelf lookup table argument over the auxiliary field $\mathbb{F}_P$ with (ii) our newly-proposed CMC PIOP, thereby obtaining LUT arguments for *arbitrary* tables over *arbitrary* residue ring $\mathbb{Z}_Q$ (Section 4.2, Theorem 2). We provide a theoretical comparison with representative ring-friendly baselines in Table 1.
3. **Two optimized instantiations for common moduli families.** While GlueLUT-g can support any moduli Q , we observe that it can be further optimized in some important special cases:
 - (a) for $Q = pq$ with distinct primes p and q , we construct GlueLUT-v1, which replaces the explicit range proof of GlueLUT with a much cheaper wrap-around-free folding check (Section 5, Theorem 3). The cost comparison between GlueLUT-g and GlueLUT-v1 is put in Table 2.
 - (b) for $Q = p^k$ (notably p can be set as 2), we construct GlueLUT-v2, which uses an auxiliary extension field $\mathbb{F}_{p^k}$ together with digit-embedding consistency checks, avoiding both the tiny exceptional set issue and the representation waste of large auxiliary primes (Section 6, Theorem 4); the comparison between GlueLUT-g and GlueLUT-v2 is given in Table 3.
4. **Prototype implementation.** We implement GlueLUT-v1 and GlueLUT-v2 as stand alone PIOPs and report prover/verifier running times and transcript sizes, validating the practical efficiency and the scaling behavior predicted by our theoretical analyses (Section 7).

1.2 Technique Overview

We consider a SNARK whose *main arithmetic* is over a residue ring $\mathbb{Z}_Q$. As in essentially all multilinear polynomial based SNARKs, the prover commits to the evaluation table of a multilinear witness polynomial $g \in \mathbb{Z}_Q[X_0, \dots, X_{\mu-1}]^{(\leq 1)}$ over the Boolean hypercube B_μ , where $B_\mu := \{0, 1\}^\mu$ and $n := 2^\mu$. A lookup

Table 1: Theoretical comparison of LUT arguments over rings ^a.

	Permutation-check based approach [7]	Multiset-check based approach [11]	GlueLUT
Prover	$n \cdot (O(\log m) + o(\log \log n))$ ^b	$O(n + m)$	$O(n + m)$
Verifier	$O(\log n + \log m)$	$O(\log(n + m))$	$O(\log(n + m))$
Proof Size	$O(\log n + \log m)$	$O(\log(n + m))$	$O(\log(n + m))$
General ^c	✓	✗	✓

•^a n denotes the witness size, and m notes the table size.

•^b In [7], the authors mentioned that the prover complexity can be improved to $n \cdot (\tilde{O}(\sqrt{\log m}) + o(\log \log n))$ when NTT is available and $m \leq n$.

•^c The general property means that the lookup argument can handle arbitrary tables over the entire ring $\mathbb{Z}_Q$.

Table 2: Comparison between GlueLUT-g and GlueLUT-v1 over $\mathbb{Z}_{pq}$ ^a.

	P's Operation	V's Complexity	Proof Size
GlueLUT-g	$\geq 4n$ sumcheck over $\mathbb{Z}_{pq}$ + $\geq 4n$ sumcheck over $\mathbb{Z}_P$ + $\geq 5n$ range proof over $\mathbb{Z}_{Ppq}$ + n lookup with size m over $\mathbb{F}_P$ ^b	$O(\log n + \log m)$	$O(\log n + \log m)$
GlueLUT-v1	n sumcheck over $\mathbb{Z}_{pq}$ + n sumcheck over $\mathbb{F}_{P'}$ + n lookup with size m over $\mathbb{F}_{P'}$ ^c	$O(\sqrt{n} + \log m)$	$O(\log n + \log m)$

•^a n denotes the witness size, m denotes the table size.

•^b P is a prime satisfying $P > pq$; Since $p \nmid Q$, the number of sumcheck instances over $\mathbb{Z}_{pq}$ and $\mathbb{Z}_P$ are both $\ell + 1$, and the number of range proof instances over $\mathbb{Z}_{Ppq}$ is $2\ell + 1$. When $p < q$, the base- p decomposition of Q has at least 3 digits, so $\ell \geq 3$.

•^c P' is a prime satisfying $P' > np^2q$.

Table 3: Comparison between GlueLUT-g and GlueLUT-v2 over $\mathbb{Z}_{p^k}$ ^a.

	P's Operation	V's Complexity	Proof Size
GlueLUT-g	kn sumcheck over $\mathbb{Z}_{p^k}$ + kn range proof over $\mathbb{Z}_P$ + n lookup with size m over $\mathbb{F}_P$ ^b	$O(\log n + \log m)$	$O(\log n + \log m)$
GlueLUT-v2	kn range proof over $\mathbb{Z}_{p^k}$ + n lookup with size m over $\mathbb{F}_{p^k}$	$O(\log n + \log m)$	$O(\log n + \log m)$

•^a n denotes the witness size, m denotes the table size.

•^b P is a prime satisfying $P > p^k$;

constraint additionally requires that each witness value $g(b)$ belongs to a public table T of size $m := 2^{\ell}$.

Our central design choice is to treat “being in a table” as a *representation-level* property (membership among canonical representatives), rather than as a

property that must be enforced using the ring algebra of $\mathbb{Z}_Q$ itself. Accordingly, we perform the lookup over a field where mature LUT arguments exist, and then connect the field witness back to the ring witness via a dedicated CMC PIOP.

A high-level picture: lookup over a field + glue to the ring. For any $a \in \mathbb{Z}_Q$, we write $\text{rep}_Q(a) \in [0, Q)$ for its canonical integer representative. When we choose a prime $P > Q$, we can view $[0, Q)$ as a subset of $\mathbb{F}_P$, and therefore we can represent each $\text{rep}_Q(a)$ as a field element.

We introduce an auxiliary multilinear polynomial $f \in \mathbb{F}_P[X_0, \dots, X_{\mu-1}]^{(\leq 1)}$ whose evaluations encode the *same integers* as g on B_μ :

$$\forall b \in B_\mu, \quad f(b) = \text{rep}_Q(g(b)) \in [0, Q) \subset \mathbb{F}_P.$$

Then the lookup can be enforced entirely in the field domain by running any standard field lookup argument on f , while the ring SNARK continues to use g for all ring arithmetic constraints. The only missing piece is a *glue proof* that certifies the above coupling of f and g .

General framework based on Cross-Modulus Consistency PIOP. Let P and Q be coprime integer and $P > Q$ and consider two multilinear polynomials f over $\mathbb{Z}_P$ and g over $\mathbb{Z}_Q$, we need a PIOP that convinces the verifier that they agree as canonical integers on the hypercube, i.e., $\forall b \in B_\mu, f(b) \equiv_{\mathbb{Z}} g(b)$, where $\equiv_{\mathbb{Z}}$ means equality of their representatives in $[0, Q)$.

We use the ring $\mathbb{Z}_{PQ}$ as a bridge to connect the two algebraic structures. For each $b \in B_\mu$, the pair $(f(b) \pmod{P}, g(b) \pmod{Q})$ determines a unique element in $\mathbb{Z}_{PQ}$ via the CRT. Concretely, we can define a conceptual multilinear polynomial

$$h(b) \in \mathbb{Z}_{PQ} \quad \text{s.t.} \quad h(b) \equiv f(b) \pmod{P}, \quad h(b) \equiv g(b) \pmod{Q}.$$

By CRT, this $h(b)$ exists and is unique in $\mathbb{Z}_{PQ}$. Our key observation is that: if in addition $h(b)$ lies in the canonical range $[0, Q)$ (as an integer), then necessarily $f(b)$ and $g(b)$ represent the same integer in $[0, Q)$. Thus, cross-modulus consistency reduces to a range proof of h .

Let p and q be the smallest prime factors of P and Q , and set $s := \min(p, q)$ and note that s is the exceptional set in $\mathbb{Z}_{PQ}$. To guarantee that all table elements used in the range proof lie in the exceptional set $[0, s)$, we enforce $h(b) < Q$ via a base- s decomposition. The subtlety is that a base- s decomposition of h alone does *not* certify $h < Q$: digits $h_i(b) \in [0, s-1]$ only guarantee that the canonical representative of $h(b)$ lies in $[0, s^\ell)$, which may strictly contain $[0, Q)$. To bridge this gap, the range constraint is enforced by proving that $Q-1-h(b)$ admits a valid base- s digit decomposition with borrow propagation, which is equivalent to $h(b) \in [0, Q-1]$.

Let $\ell := \lceil \log_s Q \rceil$ and write the base- s expansion of $Q-1$ as

$$Q-1 = \sum_{i=0}^{\ell-1} q'_i s^i, \quad q'_i \in [0, s-1].$$

To certify $h(b) \leq Q - 1$ it suffices to exhibit, for each $b \in B_\mu$, base- s digits $h_i(b), a_i(b) \in [0, s - 1]$ and borrow bits $\beta_i(b) \in \{0, 1\}$ (with $\beta_0(b) = \beta_\ell(b) = 0$) satisfying the digit-borrow relations in $\mathbb{Z}_{PQ}$:

$$h_i(b) + \beta_i(b) + a_i(b) \equiv q'_i + s \cdot \beta_{i+1}(b) \pmod{PQ}, \quad \forall i \in [\ell - 1].$$

Intuitively, $h_i(b)$ are the base- s digits of $h(b)$, $a_i(b)$ are the digits of the slack $Q - 1 - h(b)$, and $\beta_i(b)$ records the propagated borrow between digits. Since all terms are constrained to lie in $[0, 2s - 1]$ at each digit, these congruences imply the corresponding integer equalities, and summing over i yields

$$\sum_{i=0}^{\ell-1} h_i(b)s^i + \sum_{i=0}^{\ell-1} a_i(b)s^i = Q - 1,$$

which forces the canonical representative of $h(b)$ to lie in $[0, Q - 1]$.

In the protocol, no explicit commitment to an oracle over $\mathbb{Z}_{PQ}$ is required. Instead, the auxiliary polynomials are represented by their residues modulo P and modulo Q : digit polynomials $\{f_i\}$ over $\mathbb{Z}_P$ and $\{g_i\}$ over $\mathbb{Z}_Q$, together with residue polynomials $\{aP_i, \beta P_i\}$ over $\mathbb{Z}_P$ and $\{aQ_i, \beta Q_i\}$ over $\mathbb{Z}_Q$. Two **ZeroCheck** instances over $\mathbb{Z}_P$ and $\mathbb{Z}_Q$ enforce the reconstruction identities for f and g and the digit-borrow relations componentwise, while a small range proof described in [11] enforces $h_i, a_i \in [0, s - 1]$ and $\beta_i \in \{0, 1\}$ on B_μ . Whenever a value in $\mathbb{Z}_{PQ}$ is needed by the range proof verifier, it is reconstructed deterministically from its oracles modulo P and Q via CRT.

With this glue in place, the full LUT protocol for $\mathbb{Z}_Q$ is conceptually simple:

1. Encode the table $T \subseteq \mathbb{Z}_Q$ as a field table $\tilde{T} \subseteq \mathbb{F}_P$ by mapping each entry to its representative in $[0, Q)$.
2. Prove that $f(b) \in \tilde{T}$ for all $b \in B_\mu$ using any LUT argument over a *field* $\mathbb{F}_P$.
3. Run cross-modulus consistency PIOP to ensure that the field witness f is consistent with the ring witness g .

This modularity has additional benefit: improvements to LUT arguments over fields can be plugged into our framework without changing the ring side.

Specialization for $\mathbb{Z}_{pq}$: removing explicit range proofs. When $Q = pq$ with large primes $p < q$, the smallest prime factor of Q is large. In the general framework, range enforcement is implemented via base- s digits and thus becomes expensive when s is large (or when one uses table splitting to simulate a small s at the cost of extra commitments). We therefore replace the range proof step by a different consistency check that avoids the explicit range proofs.

We aim to certify that $f(b) \equiv g(b) \pmod{Q}$ for all $b \in B_\mu$, where $f(b) \in [0, Q)$ is already enforced by the lookup in $\mathbb{F}_P$. A standard way to compress many equalities into one is to take a random linear combination:

$$u := \sum_{b \in B_\mu} w_b \cdot f(b), \quad v := \sum_{b \in B_\mu} w_b \cdot g(b), \quad \text{and check} \quad u \equiv v \pmod{Q}.$$

The challenge is that u is computed in $\mathbb{F}_P$ while v is in $\mathbb{Z}_Q$. We address this by choosing P large enough so that u does not wrap around modulo P . Then the computation of u in $\mathbb{F}_P$ coincides with integer arithmetic, and reducing u modulo Q is meaningful.

We want three properties simultaneously: (i) the weights w_b are “random enough” to give soundness, and (ii) they admit an efficient proof of the claimed u and v . (iii) the u cannot be too large to avoid wrap-around modulo P . To achieve these, we factor the weight function into two halves of the hypercube. The verifier samples two multilinear polynomials $\tilde{r}$ and $\tilde{t}$ over $\mu/2$ variables by choosing their evaluation tables on $B_{\mu/2}$. Then we set

$$w_{(x,y)} := \tilde{r}(x) \cdot \tilde{t}(y), \quad (x, y) \in B_{\mu/2} \times B_{\mu/2}.$$

To avoid wrap-around and get soundness, we sample the evaluation tables of $\tilde{r}$ and $\tilde{t}$ from $[0, p)$, which lies in an exceptional set of $\mathbb{Z}_{pq}$. Since $f(b) \in [0, pq)$ (already guaranteed by the lookup table whose entries are interpreted as $[0, pq)$), we obtain an explicit upper bound on u as an integer. Choosing P larger than this bound guarantees no wrap-around. If f and g disagree on any hypercube point, then $u - v \pmod{Q}$ becomes a nonzero low-degree polynomial in the verifier randomness drawn from the exceptional set, so the generalized Schwartz–Zippel lemma yields a constant soundness gap that can be amplified by repetition.

The prover can prove the claimed values of u and v using two sumcheck instances (one over $\mathbb{F}_P$, one over $\mathbb{Z}_Q$), reducing verification to a small number of oracle openings at random points, and the verifier can compute $\tilde{r}(z)$ and $\tilde{t}(z)$ by itself. This removes the explicit range proof from the glue step and yields the specialized $\mathbb{Z}_{pq}$ protocol.

Specialization for $\mathbb{Z}_{p^k}$: auxiliary field $\mathbb{F}_{p^k}$ and digit-level consistency. For $Q = p^k$ (especially $p = 2$), exceptional-set based amplification is ineffective because the exceptional set is tiny, and the $\mathbb{Z}_{pq}$ strategy gives poor soundness. Moreover, embedding $[0, p)$ into a large $\mathbb{F}_P$ is wasteful for the general framework. We therefore choose an auxiliary field whose size matches the ring: $\mathbb{F}_{p^k}$.

Fix an $\mathbb{F}_p$ -basis $\{\omega_0, \dots, \omega_{k-1}\}$ of $\mathbb{F}_{p^k}$ and define

$$\Psi : \mathbb{Z}_{p^k} \rightarrow \mathbb{F}_{p^k}, \quad c = \sum_{j=0}^{k-1} c_j p^j \mapsto \sum_{j=0}^{k-1} c_j \omega_j,$$

where each digit $c_j \in [0, p-1]$ is the base- p representation of the canonical representative of c . Our goal is to enforce $f(b) = \Psi(g(b))$ for all $b \in B_\mu$.

The field $\mathbb{F}_p$ can be used as a bridge to connect the two algebraic structures. We decompose the ring witness polynomial into digit polynomials

$$g = \sum_{j=0}^{k-1} g_j \cdot p^j, \quad \text{with} \quad \forall b \in B_\mu, \quad g_j(b) \in [0, p-1].$$

We then introduce field polynomials $f_j \in \mathbb{F}_p[X]^{(\leq 1)}$ intended to encode the same digits:

$$\forall b \in B_\mu, \quad f_j(b) = g_j(b) \text{ as integers in } [0, p-1].$$

Finally we define the auxiliary-field witness polynomial

$$f := \sum_{j=0}^{k-1} f_j \cdot \omega_j \in \mathbb{F}_{p^k}[X_0, \dots, X_{\mu-1}]^{(\leq 1)}.$$

The lookup argument is then executed over $\mathbb{F}_{p^k}$ on f . The consistency proof has three logical components:

1. *Decomposition correctness*: certify $g = \sum_j g_j p^j$ using a random-point check (or ZeroCheck).
2. *Digit range*: certify $g_j(b) \in [0, p-1]$ for all b using a ring range proof over $\mathbb{Z}_{p^k}$ ($[0, p-1]$ is the exceptional set of $\mathbb{Z}_{p^k}$).
3. *Cross-domain digit agreement*: certify that each f_j matches g_j modulo p . This is done by sampling a random evaluation point in a suitable extension and comparing opened values under the natural reduction map Φ from the extension ring to the extension field (e.g., $GR(p^k, r) \rightarrow \mathbb{F}_{p^r}$). Conceptually, the verifier checks $f_j(\gamma) = \Phi(g_j(\gamma))$ for each j , which enforces equality of the digit polynomials. A final random-point check enforces $f = \sum_j f_j \omega_j$ in $\mathbb{F}_{p^k}$.

In this construction we tie the ring witness g to the auxiliary field witness f through their base- p digits, avoiding both the tiny exceptional set problem and the large field waste of the general framework.

1.3 Related Works

LUT arguments. LUT arguments have been extensively studied in the context of SNARKs over prime fields. A large body of work has proposed highly efficient constructions based on different algebraic principles. Early and influential approaches are based on *multiset equality*, which encodes table membership as an equality between product polynomials [3, 21, 48]. Subsequent works developed alternative paradigms, including *accumulator-based* techniques [20, 10, 28], *subvector extraction (matrix-vector) methods* [46, 39, 47, 41, 40], and *LogUp-style rational-function arguments* [26, 18, 19, 38, 8, 49, 17, 6, 9]. Hafezi et al. [27] provide a comprehensive systematization and comparison of these techniques.

More recently, Bünz et al. [7] proposed a new LUT argument construction based on permutation checks. Their approach is particularly notable for relying on sumcheck, which makes it algebraically more flexible. All of the above techniques, however, were developed in the setting of fields and are typically instantiated over large prime fields. Our work differs in focus: rather than proposing a new LUT argument over fields, we study how LUT arguments can be realized efficiently over *residue rings*.

Cascudo et al. [11] constructed range proofs over rings in the context of verifiable homomorphic encryption. Their work demonstrates that meaningful

lookup-like functionality is possible over rings when the table elements are restricted to carefully structured subsets (e.g., exceptional sets). Our work is complementary: we target general LUT arguments over residue rings that support arbitrary tables, without requiring such structural restrictions.

SNARKs over rings. A recent line of work has begun constructing SNARKs over rings. In [14], the sumcheck and GKR protocols were generalized to finite commutative rings, where the verifier’s challenges are required to be sampled from an exceptional subset of the ring. In [4], a sumcheck protocol over rings was developed and further applied to solve the Rank-1 Constraint System (R1CS) problem over rings. Moreover, the sumcheck protocol was extended to infinite non-commutative rings in [42].

The Rinocchio protocol [22] was the first complete SNARK designed for ring-based arithmetic circuits, albeit requiring a trusted setup. Subsequent works focused on transparent SNARK constructions over (Galois) rings [44,31,45,30]. These systems establish the feasibility of efficient arithmetic proofs over residue rings, but do not address LUT arguments.

Our work complements this line of research by providing the first general LUT arguments with linear prover’s complexity over residue rings, thereby filling a key missing component in the design of SNARK systems over rings.

Besides constructing SNARKs directly over rings, another line of work considers simulating ring arithmetic within field-based proof systems. Garreta et al. [23] propose Zinc, which realizes SNARKs for computations over the rational ring $\mathbb{Q}$ by expressing rational arithmetic using operations over a finite field $\mathbb{F}_p$. This simulation framework enables proof systems for computations over $\mathbb{Q}$ and, as a special case, over the integer ring $\mathbb{Z}$. Moreover, the approach can be further extended to simulate arithmetic over residue rings $\mathbb{Z}_Q$ by representing their operations through integer arithmetic and enforcing the corresponding constraints within the field-based proof system.

The Chinese Remainder Theorem in SNARKs. Several recent works employ the Chinese Remainder Theorem (CRT) to address the *Non-Naive Arithmetic* (NNA) problem in SNARKs [1,35]. The NNA problem concerns proving correctness of arithmetic operations that are not natively supported by the algebraic domain in which the SNARK is defined. These works use the CRT within a single underlying algebraic structure to express other arithmetic operations.

In contrast, our use of the CRT serves a fundamentally different purpose. We use the CRT as a *consistency mechanism* to connect witnesses represented over *different* algebraic structures. Concretely, we combine CRT with an range proof PIOP over rings to enforce that two representations correspond to the same underlying logical values. This enables lookup arguments over rings by soundly transferring membership checks to a field-based lookup protocol and then transporting the result back to the ring via consistency constraints. This role of the CRT is orthogonal to NNA focused applications and is not captured by prior CRT based SNARK constructions.

2 Preliminaries

2.1 Notions

We let λ be the security parameter. We use the abbreviation PPT to denote probabilistic polynomial-time. We denote by $x \leftarrow S$ the event where x is uniformly sampled from a set S . For $a, b \in \mathbb{N}$, we use $[a, b]$ to denote by $\{a, \dots, b\}$. For $a \in \mathbb{N}$, we use $[a]$ to denote by $\{0, 1, \dots, a\}$. We use $\mathfrak{R}$ to denote a ring. For an extension field $\mathbb{F}_{p^r}$ of a subfield $\mathbb{F}_p$, where $p \geq 2$ is a prime and $r \geq 1$ is an integer, we fix some monic, irreducible polynomial $f(X)$ of degree r and use the representation $\mathbb{F}_{p^r} \cong \mathbb{F}_p[X]/f(X)$. Every $w \in \mathbb{F}_{p^r}$ can be uniquely written as $w = \sum_{i \in [r]} w_i X^{i-1}$; we can view every element over $\mathbb{F}_{p^r}$ as a vector from $\mathbb{F}_p^r$.

2.2 Background on Rings

We denote by $\mathfrak{R}[X, \dots, X_{\ell-1}]$ the set of polynomials of ℓ variables and coefficients in $\mathfrak{R}$ and by $\mathfrak{R}^{(\leq d)}[X_0, \dots, X_{\ell-1}]$ the polynomials of individual degree $\leq d$.

Definition 1 (Exceptional sets [22]). *An exceptional set is a subset $S \subset \mathfrak{R}$ with the property that $a - b$ is an invertible element of $\mathfrak{R}$ for each pair of distinct elements $a, b \in S$.*

Lemma 1 (Generalized Schwartz–Zippel lemma [22]). *Let $f \in \mathfrak{R}[X_1, \dots, X_\ell]$ be a non-zero polynomial of total degree d . Then for an exceptional set $S \subset \mathfrak{R}$,*

$$\Pr_{(r_1, \dots, r_\ell) \leftarrow S^\ell} [f(r_1, \dots, r_\ell) = 0] \leq \frac{d}{|S|}.$$

Definition 2 (Galois ring [43]). *Let p be a prime and $s, r \geq 1$ be integers. Let $f(X) \in \mathbb{Z}_{p^s}[X]$ be a monic polynomial of degree r whose reduction $\bar{f}(X) \in \mathbb{F}_p[X]$ (obtained by reducing coefficients modulo p) is irreducible over $\mathbb{F}_p$. The Galois ring $GR(p^s, r)$ is defined as $GR(p^s, r) := \mathbb{Z}_{p^s}[X]/(f(X))$. It is a finite commutative ring of characteristic p^s and cardinality p^{sr} .*

Fact 1 (Reduction modulo p and induced map) *Let p be a prime and let $s \geq 1$. Let $\rho : \mathbb{Z}_{p^s} \rightarrow \mathbb{F}_p$ be the natural reduction map $\rho(a) = a \bmod p$. Then ρ is a surjective ring homomorphism with $\ker(\rho) = (p)$. It extends coefficient-wise to a surjective ring homomorphism $\rho : \mathbb{Z}_{p^s}[x] \rightarrow \mathbb{F}_p[x]$ with $\ker(\rho) = (p)$.*

Moreover, for any $h(x) \in \mathbb{Z}_{p^s}[x]$, ρ induces a surjective homomorphism

$$\bar{\rho} : \mathbb{Z}_{p^s}[x]/(h(x)) \twoheadrightarrow \mathbb{F}_p[x]/(\bar{h}(x)),$$

where $\bar{h}(x) := \rho(h(x))$. If $\bar{h}(x)$ is irreducible over $\mathbb{F}_p$ and $\deg(\bar{h}) = r$, then $\mathbb{F}_p[x]/(\bar{h}(x)) \cong \mathbb{F}_{p^r}$.

Besides these definitions, we also provide some commonly used PIOPs over rings in Appendix A.

2.3 SNARKs

Let $\mathcal{R}$ be an efficiently computable binary relation which consists of pairs of the form (x, w) where x is a statement and w is a witness. Let $\mathcal{L}$ be the language associated with the relation $\mathcal{R}$, i.e. $\mathcal{L} = \{x | \exists w, \text{ s.t. } \mathcal{R}(x, w) = 1\}$.

A proof or argument system for $\mathcal{R}$ consists of a triple of PPT algorithms $\Pi = (\text{Setup}, \text{Prove}, \text{Verify})$ defined as follows:

- $\text{Setup}(1^\lambda) \rightarrow (\sigma, \text{vk})$: take a security parameter λ and outputs a common (structured) reference string σ together with private verification information vk .
- $\text{Prove}(\sigma, x, w) \rightarrow \pi$: on input σ , a statement x and witness w , outputs an argument π .
- $\text{Verify}(\sigma, \text{vk}, x, \pi) \rightarrow 1/0$: on input σ , the private verification key vk , a statement x and a proof π , it outputs either 1 indicating accepting the argument or 0 for rejecting it.

Definition 3 (SNARK [22]). *A triple of polynomial time algorithms $(\text{Setup}, \text{Prove}, \text{Verify})$ is a SNARK for an NP relation $\mathcal{R}$, if the following properties are satisfied:*

- *Completeness. For all $(x, w) \in \mathcal{R}$, the following holds:*

$$\Pr \left[\begin{array}{l} (\sigma, \text{vk}) \leftarrow \text{Setup}(1^\lambda); \pi \leftarrow \text{Prove}(\sigma, x, w) : \\ \text{Verify}(\sigma, \text{vk}, x, \pi) = 1 \end{array} \right] = 1.$$

- *Knowledge Soundness. For any PPT adversary $\mathcal{A}$, there exists a PPT algorithm $\mathcal{E}_{\mathcal{A}}$ such that the following probability is negligible in λ :*

$$\Pr \left[\begin{array}{l} (\sigma, \text{vk}) \leftarrow \text{Setup}(1^\lambda); ((\tilde{x}, \tilde{\pi}); w') \leftarrow \mathcal{A} | \mathcal{E}_{\mathcal{A}}(\sigma) : \\ \text{Verify}(\sigma, \text{vk}, \tilde{x}, \tilde{\pi}) = 1 \wedge \mathcal{R}(\tilde{x}, w') = 0 \end{array} \right]$$

- *A non-interactive argument of knowledge satisfies knowledge is succinct if the size of proof π and the time to Verify are sublinear in the size of the statement proven.*

If a SNARK does not require a private verification key vk , then the SNARK scheme is referred to as a public-key SNARK. If a SNARK does not require a CRS σ , then the SNARK scheme is referred to as transparent.

2.4 Polynomial Commitment Scheme

Definition 4 (Polynomial Commitment Scheme [25].). *A polynomial commitment scheme for multilinear polynomial over Ring $\mathfrak{R}$ is a tuple of four protocols $PC = (\text{Gen}, \text{Commit}, \text{Open}, \text{Eval})$:*

- $pp \leftarrow \text{Gen}(1^\lambda, \mu)$ takes as input μ (the number of variables in a multilinear polynomial); produces public parameters pp .
- $\mathcal{C} \leftarrow \text{Commit}(pp, \mathcal{G}, \gamma)$: takes any input a μ -variate multilinear polynomial over a Ring $\mathcal{G} \in \text{Poly}(\mathfrak{R})$ and a random string γ produces a commitment $\mathcal{C}$.

- $b \leftarrow \text{Open}(pp, \mathcal{C}, \mathcal{G}, \gamma)$: verifies the opening of commitment $\mathcal{C}$ to the μ -variate multilinear polynomial $\mathcal{G} \in \text{Poly}(\mathfrak{R})$; outputs $b \in \{0, 1\}$.
- $b \leftarrow \text{Eval}(pp, \mathcal{C}, z, v, \mu, \mathcal{G})$ is a protocol between a PPT prover $\mathcal{P}$ and verifier $\mathcal{V}$. Both $\mathcal{V}$ and $\mathcal{P}$ hold a commitment $\mathcal{C}$, the number of variables μ , a scalar $v \in \mathfrak{R}$ and $z \in \mathfrak{R}^\mu$. $\mathcal{P}$ additionally knows a μ -variate multilinear polynomial $\mathcal{G} \in \text{Poly}(\mathfrak{R})$. $\mathcal{P}$ attempts to convince $\mathcal{V}$ that $\mathcal{G}(z) = v$. At the end of the protocol, $\mathcal{V}$ outputs $b \in \{0, 1\}$.

A tuple of four protocol $(\text{Gen}, \text{Commit}, \text{Open}, \text{Eval})$ is an extractable polynomial commitment scheme for multilinear polynomials over Ring $\mathfrak{R}$ if the following conditions hold.

- **Completeness.** For any multilinear polynomial $\mathcal{G} \in \text{Poly}(\mathfrak{R})$,

$$\Pr \left[\begin{array}{l} pp \rightarrow \text{Gen}(1^\lambda, \mu); \mathcal{C} \leftarrow \text{Commit}(pp, \mathcal{G}, \gamma) : \\ \text{Eval}(pp, \mathcal{C}, r, v, \mu, \mathcal{G}) = 1 \wedge v = \mathcal{G}(r) \end{array} \right] \geq 1 - \text{negl}(\lambda)$$

- **Binding.** For any PPT adversary $\mathcal{A}$, size parameter $\mu \geq 1$,

$$\Pr \left[\begin{array}{l} pp \leftarrow \text{Gen}(1^\lambda, \mu); \mathcal{C}, \mathcal{G}_0, \mathcal{G}_1 \leftarrow \mathcal{A}(pp); \\ b_0 \leftarrow \text{Open}(pp, \mathcal{C}, \mathcal{G}_0, \gamma); b_1 \leftarrow \text{Open}(pp, \mathcal{C}, \mathcal{G}_1, \gamma); \\ b_0 = b_1 \neq 0 \wedge \mathcal{G}_0 \neq \mathcal{G}_1 \end{array} \right] \leq \text{negl}(\lambda)$$

- **Knowledge soundness.** Eval is a succinct argument of knowledge for the following NP relation given $pp \leftarrow \text{Gen}(1^\lambda, \mu)$.

$$\mathcal{R}_{\text{Eval}}(pp) = \{((\mathcal{C}, z, v), (\mathcal{G})) : \mathcal{G} \in GR(p^s, r)[\mu] \wedge \mathcal{G}(z) = v \wedge \text{Open}(pp, \mathcal{C}, \mathcal{G}, \gamma) = 1\}$$

There are several works focus on constructing SNARKs over Galois rings [44, 31] [45, 30] and residue rings [11]. These schemes support opening of multilinear polynomials at extension ring elements.

3 Starting Point: Why LUT Arguments over Fields Cannot be Directly Extended to Rings

In this section, we identify the main obstacles when adapting state-of-the-art LUT arguments from fields to rings. Our discussion follows the taxonomy surveyed by Hafezi et al. [27], and also considers permutation-check based LUT arguments such as those introduced by Bünz et al. [7].

3.1 Multiset-check based approach

The multiset equality check [34] is a basic ingredient in many LUT arguments and appears in wide range of constructions [3, 21, 48]. Informally, it reduces a lookup table claim to checking whether two (multi)sets are equal. Given two sets $A = \{a_0, a_1, \dots, a_{n-1}\}$ and $B = \{b_0, b_1, \dots, b_{n-1}\}$, we first define the polynomials

$\Gamma_A(x) = \prod_{i=0}^{n-1} (x - a_i)$ and $\Gamma_B(x) = \prod_{i=0}^{n-1} (x - b_i)$. Over a field, $\mathbb{F}[X]$ is a unique factorization domain, and monic factorizations into linear factors are unique. Hence $\Gamma_A(X) = \Gamma_B(X)$ implies $A = B$ as multisets. Therefore, a verifier can test multiset equality by randomly sampling a challenge x and checking whether $\Gamma_A(x) = \Gamma_B(x)$ using a “grand product” argument.

However, this mapping from sets to polynomials is no longer injective over rings. Equality of the product polynomials only implies equality of the underlying sets when all elements in the sets are drawn from the exceptional set of the ring. The proof of Fact 2 is given in Appendix B.1.

Fact 2 (Injectivity of the product encoding over an exceptional set) *Let $\mathfrak{R}$ be a commutative ring with identity $1 \neq 0$, and let $S \subseteq \mathfrak{R}$ be an exceptional set. For any finite multiset A over S , define the monic polynomial*

$$\Gamma_A(X) := \prod_{a \in A} (X - a) \in \mathfrak{R}[X].$$

Then the map $A \mapsto \Gamma_A(X)$ is injective on multisets supported on $\mathcal{E}$: for any two finite multisets $A, B \subset S$,

$$\Gamma_A(X) = \Gamma_B(X) \text{ in } \mathfrak{R}[X] \implies A = B \text{ as multisets.}$$

Therefore, over rings, multiset equality-based LUT arguments are sound only when all table and witness elements lie in an exceptional set. This suffices for some structured tables (e.g., digit tables used in certain range proofs in Cascudo et al. [11]). When table elements cannot be confined to the exceptional set, this technique becomes inapplicable. We now present Lemma 2 and Lemma 3 illustrating this issue over the rings $\mathbb{Z}_{pq}$ and $\mathbb{Z}_{p^k}$. The proofs of these lemmas are given in Appendix B.2 and Appendix B.3 respectively.

Lemma 2. *Let p and q be prime numbers. Then there exist distinct subsets $A, B \subseteq \mathbb{Z}_{pq}$ with $A \neq B$ such that $\prod_{a \in A} (x - a) = \prod_{b \in B} (x - b)$ as polynomials over $\mathbb{Z}_{pq}$.*

Lemma 3. *Let p be a prime and k a positive integer. For $p \geq 5$ and $k \geq 2$, or for $p \in \{2, 3\}$ and $k \geq 3$, there exist distinct subsets $A, B \subseteq \mathbb{Z}_{p^k}$ with $A \neq B$ such that $\prod_{a \in A} (x - a) = \prod_{b \in B} (x - b)$ as polynomials over $\mathbb{Z}_{p^k}$.*

3.2 LogUp-based approach

LogUp-style LUT arguments [26, 18, 19, 38, 8] [49, 17, 6, 9] reduce lookup correctness to an identity of rational functions evaluated at verifier challenges. At a conceptual level, these identities are algebraic reformulations of multiset equality: they enforce that two multisets induce the same product polynomial (or the same logarithmic derivative thereof) at random challenges. Consequently, any setting in which the product encoding fails to be injective also admits indistinguishable but unequal multisets, and therefore the LogUp soundness analysis does not extend to general rings without additional restrictions (e.g., confinement to exceptional sets).

3.3 Subvector extraction (matrix-vector) based approach

Hafezi et al. [27] also surveyed LUT arguments based on the subvector extraction technique. In this approach, a lookup is encoded as a matrix-vector multiplication, typically a sparse rank one constraint system, and the prover shows that the selected entries of a public table are consistent with the claimed witnesses.

Existing representative constructions in this category, including Caulk [46], Caulk+ [39], and Baloo [47], are all built on the KZG polynomial commitment scheme [32] and therefore rely on pairing structures. At present, we are not aware of pairing based polynomial commitments over general rings, so these approaches do not directly extend to rings.

The later protocol Lasso [41] supports arbitrary polynomial commitments. Its sparse matrix commitments rely on the Spark technique, which reduces the problem to memory checking. In standard instantiations, memory checking is implemented via algebraic checks that ultimately reduce to multiset- or permutation-style consistency; thus, the same ring-level obstructions reappear unless one can instantiate these sub-components efficiently over rings.

More recently, Shout [40] proposes an alternative memory checking method. It requires a Spark-naïve polynomial commitment scheme, i.e., a PCS in which committing to zero values is essentially free. Known realizations of Spark-naïve commitments rely on group-based assumptions (e.g., DDH-type structures or pairings), and to the best of our knowledge no existing instantiation provides this functionality over rings.

3.4 Accumulator-based approach

One can also construct LUT arguments using cryptographic accumulators that support commit-and-prove batch openings. These accumulators all rely on pairing-based cryptography [20] or RSA accumulators [10,28], in which the table elements must be encoded with prime numbers. As such, these techniques do not directly extend to rings.

3.5 Permutation-check based approach

Bünz et al. [7] introduce two permutation check arguments (BiPerm and MulPerm) and show how to derive LUT arguments from them. A direct instantiation over rings faces two obstacles. First, the linear-time prover of BiPerm relies on Spark-naïve polynomial commitments, which are not currently available over rings. Second, MulPerm can be instantiated with an arbitrary PCS, but its fastest prover implementations rely on NTT-accelerated to speed up the per-round computations in sumcheck. Over many rings, suitable NTTs do not exist, and without them the prover time increases to $O(n \log n)$. Taken together, these issues make permutation-check based LUT arguments significantly less efficient over rings.

Chen et al. [12] also propose a LUT argument based on permutation-check; their construction assumes Spark-naïve polynomial commitments and faces the same barrier in the ring setting.

4 GlueLUT: Lookups over an Auxiliary field with a Cross-Modulus Consistency Check

In this section, we present a generic approach for supporting lookup constraints in SNARKs whose *main arithmetic* is carried out over a residue ring $\mathbb{Z}_Q$. The key idea is to enforce lookup constraints over a suitably large auxiliary field $\mathbb{F}_P$, while keeping the algebraic circuit constraints over $\mathbb{Z}_Q$, and to connect the two witnesses via a cross-modulus consistency check.

4.1 Cross-Modulus Consistency PIOP

In this subsection, we introduce a general-purpose polynomial IOP for enforcing *cross-modulus consistency* between witnesses represented modulo two coprime integers. This PIOP addresses the core difficulty that arises when different components of a proof system operate over different algebraic structures, and it serves as the main technical glue in our lookup constructions over rings. We begin by formalizing the notion of integer consistency between residue representations.

Notation. For $\mu \geq 1$, let $B_\mu := \{0, 1\}^\mu$ and $n := |B_\mu| = 2^\mu$. For $\iota \geq 1$, let $B_\iota := \{0, 1\}^\iota$ and $m := |B_\iota| = 2^\iota$.

Definition 5 (Canonical representative). Let $M \geq 2$ be an integer. Define the canonical representative map $\text{rep}_M : \mathbb{Z}_M \rightarrow [0, M)$ that maps each residue class to its unique integer representative in $\{0, 1, \dots, M - 1\}$.

Definition 6 (Cross-modulus consistency). Let $P, Q \geq 2$ be coprime integers. Let $f : B_\mu \rightarrow \mathbb{Z}_P$ and $g : B_\mu \rightarrow \mathbb{Z}_Q$. We say that f and g are cross-modulus consistent if $\forall b \in B_\mu, \text{rep}_P(f(b)) = \text{rep}_Q(g(b))$ as integers. We denote this relation by $\text{Cons}_{P,Q}(f, g) = 1$.

We construct a cross-modulus consistency PIOP based on the following fact 3. We show the proof of Fact 3 in the Appendix B.4.

Fact 3 Let P, Q be two coprime integers and $P > Q$. Let $x \in [0, P)$, $y \in [0, Q)$, and $z \in [0, PQ)$. If

$$z \equiv x \pmod{P}, \quad z \equiv y \pmod{Q}, \quad \text{and} \quad z \in [0, Q),$$

then $x = z$ and $y = z$ as integers. In particular, $x = y$ as integers in $[0, Q)$.

We now give a PIOP that proves $\text{Cons}_{Q,P}(f, g) = 1$ for multilinear oracles f and g . Intuitively, we define an *implicit* oracle $h : B_\mu \rightarrow \mathbb{Z}_{PQ}$ by CRT so that $h(b) \equiv f(b) \pmod{P}$ and $h(b) \equiv g(b) \pmod{Q}$ for all $b \in B_\mu$. Then $\text{Cons}_{Q,P}(f, g) = 1$ reduces to the rang proof $h(b) \in [0, Q)$ for all $b \in B_\mu$ by Fact 3. Let p and q be the smallest prime factors of P and Q , respectively, and set $s := \min(p, q)$. Then $[0, s - 1]$ is an exceptional set of $\mathbb{Z}_{PQ}$. We enforce the range condition $h(b) \in [0, Q)$ by base- s digit decomposition and Fact 4.

Cross-Modulus Consistency PIOP

- **Common input.** Coprime integers $Q < P$ and μ (so $n = 2^\mu$). Let p, q be the smallest prime factors of P, Q and set $s := \min(p, q)$. Let $\ell = \lceil \log_s(Q) \rceil$.
- **Oracles.** V has oracle access to commitments $[[f]]_P$ and $[[g]]_Q$ for multilinear polynomials $f \in \mathbb{Z}_P[X_0, \dots, X_{\mu-1}]^{(\leq 1)}$ and $g \in \mathbb{Z}_Q[X_0, \dots, X_{\mu-1}]^{(\leq 1)}$.

1. Commitment phase.

- (a) P sends commitments $\{([f_i]]_P, [[g_i]]_Q\}_{i=0}^{\ell-1}$ to multilinear polynomials $f_i \in \mathbb{F}_P[X_0, \dots, X_{\mu-1}]^{(\leq 1)}$ and $g_i \in \mathbb{Z}_Q[X_0, \dots, X_{\mu-1}]^{(\leq 1)}$. Intuitively, these are intended to be the base- s digits of f and g (in canonical representatives).
- (b) P decomposes $Q - 1 - h$ in base s as $Q - 1 - h = \sum_{i=0}^{\ell-1} a_i s^i$ with $a_i \in \mathbb{Z}_{PQ}[X_0, \dots, X_{\mu-1}]^{(\leq 1)}$, and sends commitments $\{([aP_i]]_P, [[aQ_i]]_Q\}_{i=0}^{\ell-1}$ to multilinear polynomials $a_i \in \mathbb{Z}_{PQ}[X_0, \dots, X_{\mu-1}]^{(\leq 1)}$, where aP_i and aQ_i are the RNS components of a_i modulo P and Q , respectively.
- (c) P computes borrow polynomials

$$\beta_{i+1} = \begin{cases} 1, & \text{if } h_i + \beta_i + a_i \geq q'_i + s, \\ 0, & \text{if } h_i + \beta_i + a_i < q'_i + s, \end{cases} \quad \text{with } \beta_0 = 0.$$

P sends commitments $\{([\beta P_i]]_P, [[\beta Q_i]]_Q\}_{i=0}^{\ell}$ to multilinear polynomials $\beta_i \in \mathbb{Z}_{PQ}[X_0, \dots, X_{\mu-1}]^{(\leq 1)}$, where βP_i and βQ_i are the RNS components of β_i modulo P and Q , respectively.

2. Decomposition checks.

- (a) Over $\mathbb{Z}_Q$, run **ZeroCheck** to prove $\forall b \in B_\mu$:

$$g(b) = \sum_{i=0}^{\ell-1} g_i(b) \cdot s^i;$$

$$g_i(b) + \beta Q_i(b) + aQ_i(b) = q'_i + s \cdot \beta Q_{i+1}(b) \quad \forall i \in [\ell - 1];$$

- (b) Over $\mathbb{Z}_P$, run **ZeroCheck** to prove $\forall b \in B_\mu$:

$$f(b) = \sum_{i=0}^{\ell-1} f_i(b) \cdot s^i;$$

$$f_i(b) + \beta P_i(b) + aP_i(b) = q'_i + s \cdot \beta P_{i+1}(b) \quad \forall i \in [\ell - 1];$$

where q'_i are the base- s digits of $Q - 1$.

Fig. 1: Cross-Modulus Consistency PIOP

Cross-Modulus Consistency PIOP (cont.)

3. **Range proof.**

P and V run a range proof PIOP over $\mathbb{Z}_{PQ}$ in [11] to prove that $\forall b \in B_\mu$,

- (a) $h_i(b) \in [0, s-1]$ for all $i \in [\ell-1]$;
- (b) $a_i(b) \in [0, s-1]$ for all $i \in [\ell-1]$;
- (c) $\beta_i(b) \in \{0, 1\}$ for all $i \in [1, \ell-1]$;
- (d) $\beta_0(b) = 0$ and $\beta_\ell(b) = 0$.

When V needs to query the oracles over $\mathbb{Z}_{PQ}$, it reconstructs the answers from the oracles over $\mathbb{Z}_P$ and $\mathbb{Z}_Q$.

Fig. 1: Cross-Modulus Consistency PIOP (continued)

Fact 4 Let $P, Q \geq 2$ be coprime integers, let $N = PQ$, and let s be the smallest prime divisor of N . Let $\ell = \lceil \log_s Q \rceil$, and write the base- s expansion

$$Q - 1 = \sum_{i=0}^{\ell-1} q'_i s^i, \quad q'_i \in \{0, 1, \dots, s-1\}.$$

Let $h \in \mathbb{Z}_N$. Suppose there exist integers $h_0, \dots, h_{\ell-1}$, $a_0, \dots, a_{\ell-1}$, and $\beta_0, \dots, \beta_\ell$ in $\{0, 1, \dots, N-1\}$ such that:

1. $h \equiv \sum_{i=0}^{\ell-1} h_i s^i \pmod{N}$ with $h_i \in \{0, 1, \dots, s-1\}$ for all i ;
2. $a_i \in \{0, 1, \dots, s-1\}$ for all i , and $\beta_i \in \{0, 1\}$ for all i , with $\beta_0 = \beta_\ell = 0$;
3. for every $i = 0, \dots, \ell-1$: $h_i + \beta_i + a_i \equiv q'_i + s \cdot \beta_{i+1} \pmod{N}$.

Then $h \in [0, Q-1]$ as an integer.

The proof of Fact 4 is given in the Appendix B.5.

According to Fact 4, in order to prove that $h(b) \in [0, Q-1]$ for all $b \in B_\mu$, it suffices to prove the existence of multilinear oracles

$$h_i(b) \in [0, s-1], \quad a_i(b) \in [0, s-1] \quad (i \in [\ell-1]), \quad \beta_j(b) \in \{0, 1\} \quad (j \in [\ell])$$

such that $\beta_0(b) = \beta_\ell(b) = 0$ for all $b \in B_\mu$ and the following digit-borrow constraints hold for all $b \in B_\mu$ and all $i \in [\ell-1]$: $h_i(b) + \beta_i(b) + a_i(b) \equiv q'_i + s \cdot \beta_{i+1}(b) \pmod{PQ}$, where $Q-1 = \sum_{i=0}^{\ell-1} q'_i s^i$ is the base- s expansion of $Q-1$.

In the resulting PIOP, the prover commits to the auxiliary oracles $\{h_i\}_i$, $\{a_i\}_i$, and $\{\beta_i\}_i$ over $\mathbb{Z}_{PQ}$. Crucially, we do not require an explicit oracle over $\mathbb{Z}_{PQ}$: by CRT, each committed oracle over $\mathbb{Z}_{PQ}$ can be represented by a pair of oracles over $\mathbb{Z}_P$ and $\mathbb{Z}_Q$. Accordingly, the PIOP is executed over the $\mathbb{Z}_{PQ}$, and whenever the verifier needs a value in $\mathbb{Z}_{PQ}$, it reconstructs it uniquely from the two residues via CRT. The full CMC PIOP is given in Figure 1.

Theorem 1. Let P, Q be coprime integers with $Q < P$. Let p and q be the smallest prime factors of P and Q , respectively, and set $s := \min(p, q)$. Let $n = 2^\mu$ and $\ell := \lceil \log_s Q \rceil$. Assume:

- the ZeroCheck PIOP over $\mathfrak{R}$ for μ -variate and d -degree statements has knowledge soundness error $\delta_{\mathfrak{R}, \text{Zero}}^{n,d}$;
- the range proof PIOP over $\mathbb{Z}_{PQ}$ for n values in range $[0, s-1]$ has knowledge soundness error $\delta_{\mathbb{Z}_{PQ}, \text{range}}^{n,s}$.

Then the Cross-Modulus Consistency PIOP in Figure 1:

1. has perfect completeness;
2. has knowledge soundness error at most $(\ell+1) \cdot \delta_{\mathbb{Z}_P, \text{Zero}}^{n,1} + (\ell+1) \cdot \delta_{\mathbb{Z}_Q, \text{Zero}}^{n,1} + 2\delta_{\mathbb{Z}_{PQ}, \text{range}}^{\ell n, s} + \delta_{\mathbb{Z}_{PQ}, \text{range}}^{\ell n, 2}$
3. has PIOP complexity (excluding oracle commitment costs):
 - prover complexity $O(n)$ ring operations;
 - verifier complexity $O(\log n)$ ring operations;
 - proof size $O(\log n)$ ring elements.

Proof. Completeness. Completeness follows directly from the construction.

Knowledge Soundness. We start by recalling the standard fact that sound PIOPs are knowledge sound for oracle relations (cf. HyperPlonk [13] Lemma 2.3): for a δ -sound PIOP for an oracle relation $\mathcal{R}$, if every witness consists solely of (bounded-degree) multivariate polynomials and the instance provides oracle access to these witness polynomials, then the same protocol is δ -knowledge sound.

In our PIOP, the witness is exactly a collection of polynomial oracles that are already part of the instance, hence it suffices to prove δ -soundness as above; δ -knowledge soundness follows immediately from the lemma.

Since the witness of the protocol consists solely of polynomial oracles, and the verifier interacts with the prover only through these oracles, standard results for polynomial IOPs imply that soundness directly lifts to knowledge soundness with the same error. (Details in Lemma 2.3 of HyperPlonk [13]). It therefore suffices to bound the soundness error.

Assume $\text{Cons}_{Q,P}(f, g) \neq 1$. Then there exists $b^* \in B_\mu$ such that $\text{rep}_P(f(b^*)) \neq \text{rep}_Q(g(b^*))$. Let h be the value/function over $\mathbb{Z}_{PQ}$ implicitly defined pointwise by CRT from (f, g) . By Fact 3, for this b^* at least one of the following must hold: (i) $h(b^*) \notin [0, Q-1]$, (ii) $h(b^*) \not\equiv f(b^*) \pmod{P}$, (iii) $h(b^*) \not\equiv g(b^*) \pmod{Q}$. Since CRT defines h uniquely from (f, g) , cases (ii) and (iii) cannot occur; hence $h(b^*) \notin [0, Q-1]$. According to Fact 4, if $h(b^*) \notin [0, Q-1]$, then at least one of the following holds:

1. $h(b^*) \neq \sum_{i=0}^{\ell-1} h_i(b^*) \cdot s^i$;
2. $h_{i^*}(b^*) + \beta_{i^*}(b^*) + a_{i^*}(b^*) \not\equiv q'_{i^*} + s \cdot \beta_{i^*+1}(b^*) \pmod{PQ}$ for some $i^* \in [\ell-1]$;
3. $h_{i^*}(b^*) \notin [0, s-1]$ for some $i^* \in [\ell-1]$;
4. $a_{i^*}(b^*) \notin [0, s-1]$ for some $i^* \in [\ell-1]$;
5. $\beta_{i^*}(b^*) \notin \{0, 1\}$ for some $i^* \in [1, \ell-1]$;
6. $\beta_0(b^*) \neq 0$ or $\beta_\ell(b^*) \neq 0$.

If condition (1) or (2) fails, then at least one ZeroCheck instance in Step 2 accepts a false claim. By the union bound, this happens with probability at most $(\ell+1) \cdot \delta_{\mathbb{Z}_P, \text{Zero}}^{n,1} + (\ell+1) \cdot \delta_{\mathbb{Z}_Q, \text{Zero}}^{n,1}$.

Otherwise, the decomposition identities hold. If condition (3), (4), or (5) holds, then the range proof in Step 3 must accept a false range statement. By the union bound, this happens with probability at most $2\delta_{\mathbb{Z}_{PQ}, \text{range}}^{\ell n, s} + \delta_{\mathbb{Z}_{PQ}, \text{range}}^{\ell n, 2}$.

Condition (6) is easily verified, since β_0 and β_ℓ can be computed directly by the verifier as 0 without being committed by the prover.

In summary, according to the union bound, the total soundness error is at most $(\ell + 1) \cdot \delta_{\mathbb{Z}_P, \text{Zero}}^{n, 1} + (\ell + 1) \cdot \delta_{\mathbb{Z}_Q, \text{Zero}}^{n, 1} + 2\delta_{\mathbb{Z}_{PQ}, \text{range}}^{\ell n, s} + \delta_{\mathbb{Z}_{PQ}, \text{range}}^{\ell n, 2}$.

PIOP Complexity. We count only PIOP work and messages, excluding oracle commitment costs as in HyperPlonk. The protocol runs $2\ell + 2$ ZeroCheck instances over n points (one over $\mathbb{Z}_P$, one over $\mathbb{Z}_Q$), and two range proof over ℓn values with range size s and one range proof over ℓn values with range size 2. For ZeroCheck, the prover cost is $O(n)$ and verifier cost is $O(\log n)$ over each ring. And the proof size is $O(\log n)$ ring elements.

For the range proof, the method of Cascudo et al. [11] is based on memory checking and the prover cost is $O(\ell \cdot n + s)$ ring operations, the verifier cost is $O(\log(\ell \cdot n + s))$ ring operations, the proof size is $O(\log(\ell \cdot n + s))$ ring elements.

Since ℓ and s are constants depending on P, Q and independent of the size n , the prover complexity, verifier complexity, and proof size can be simplified to $O(n)$, $O(\log n)$, and $O(\log n)$, respectively. $\square$

4.2 GlueLUT-g: The General Framework for LUT over Rings

In this section, we present GlueLUT-g, a general framework for supporting lookup tables in SNARKs whose main arithmetic is carried out over a residue ring $\mathbb{Z}_Q$. Section 3 explains why standard lookup paradigms over fields do not directly extend to rings. To bypass these obstacles, we separate lookup constraints from ring arithmetic: lookup constraints only assert membership in a table and do not require ring multiplication beyond representing elements.

Concretely, we consider a proof system whose main arithmetic is over the ring $\mathbb{Z}_Q$. Lookup constraints only assert membership in a table and do not require ring arithmetic beyond representing elements. We therefore enforce lookup constraints in an auxiliary field $\mathbb{F}_P$, where $P > Q$ ensures that all elements of $\mathbb{Z}_Q$ has a canonical representation in $\mathbb{F}_P$. The proof runs the lookup protocol in $\mathbb{F}_P$ while keeping the algebraic constraints in $\mathbb{Z}_Q$. A separate glue proof enforces that the witness in $\mathbb{F}_P$ is consistent with the witness in $\mathbb{Z}_Q$. This construction presents a proof system that supports lookup tables over the ring $\mathbb{Z}_Q$. We now present GlueLUT-g in Figure 2.

Definition 7 (Lookup Relation over $\mathbb{Z}_Q$). Let Q an integer. We define the relation $\mathfrak{R}_{\text{LUTR}}$ as the set of all tuples $(\mathbf{x}, \mathbf{w}) = ((Q, [[f]]_Q, T, [[T]]_Q); (g))$, where $g \in \mathbb{Z}_Q[X_0, \dots, X_{\mu-1}]^{(\leq 1)}$ and $T \in \mathbb{Z}_Q[X_0, \dots, X_{\iota-1}]^{(\leq 1)}$, such that for every $b \in B_\mu$, there exists an index $b' \in B_\iota$ satisfying $g(b) = T(b')$ over $\mathbb{Z}_Q$.

Theorem 2. Let p and q be the smallest prime factors of P and Q , respectively, and set $s := \min(p, q)$. Let $n = 2^\mu$ and $\ell := \lceil \log_s Q \rceil$. Assume:

GlueLUT-g: Lookup Table PIOP for $\mathbb{Z}_Q$

- **Preprocessing.** T is a lookup table with elements from $\mathbb{Z}_Q$ with size $m = 2^\ell$. P is a prime such that $P > Q$. We define the lookup table polynomial $\tilde{T} \in \mathbb{F}_P[X_0, \dots, X_{\ell-1}]^{(\leq 1)}$ where for every $a \in [0, m-1]$, $\text{rep}_P(\tilde{T}(a)) = \text{rep}_Q(T[a])$.
- 1. **Commitment Phase.**
 - (a) P commits to a multilinear polynomial $f \in \mathbb{F}_P[X_0, \dots, X_{\mu-1}]^{(\leq 1)}$ such that for every $b \in B_\mu$, $\text{rep}_P(f(b)) = \text{rep}_Q(g(b))$, where $g \in \mathbb{Z}_Q[X_0, \dots, X_{\mu-1}]^{(\leq 1)}$ is the multilinear polynomial committed in the algebraic proof over $\mathbb{Z}_Q$.
- 2. **Lookup Proof over $\mathbb{F}_P$.** P and V execute a standard lookup proof over the field $\mathbb{F}_P$ to prove that for every $b \in B_\mu$, there exists an $b' \in B_\ell$ such that $f(b) = \tilde{T}(b')$.
- 3. **Cross-Modulus Consistency Proof.** P and V execute the Cross-Modulus Consistency PIOP (Figure 1) to prove that for every $b \in B_\mu$, $\text{rep}_P(f(b)) = \text{rep}_Q(g(b))$.

Fig. 2: GlueLUT-g: Lookup Table PIOP for $\mathbb{Z}_Q$

- the lookup argument over $\mathbb{F}_P$ for vector length n and table size m has perfect completeness and knowledge soundness error $\delta_{\mathbb{F}_P, \text{LUT}}^{n,m}$;
- the Cross-Modulus Consistency PIOP between $\mathbb{F}_P$ and $\mathbb{Z}_Q$ for vector length n has perfect completeness and knowledge soundness error $\delta_{\mathbb{F}_P, \mathbb{Z}_Q, \text{CMCR}}^n$.

Then GlueLUT-g in Figure 2:

1. has perfect completeness;
2. has knowledge error at most $\delta_{\mathbb{F}_P, \text{LUT}}^{n,m} + \delta_{\mathbb{F}_P, \mathbb{Z}_Q, \text{CMCR}}^n$.
3. has PIOP complexity (excluding oracle commitment costs):
 - prover complexity $O(n + m)$ ring operations;
 - verifier complexity $O(\log n + \log m)$ ring operations;
 - proof size $O(\log n + \log m)$ ring elements.

Proof. The completeness, knowledge soundness and complexity follow directly from the LUT PIOP over $\mathbb{F}_P$ ⁷ and the Cross-Modulus Consistency PIOP. $\square$

5 GlueLUT-v1: Efficient LUT Arguments over $\mathbb{Z}_{pq}$ without Range Proofs

In Section 4.2, we presented a general lookup table PIOP framework for the ring $\mathbb{Z}_Q$ for any modulus Q . In this section, we propose a more efficient lookup table

⁷ The complexity here is based on the complexity of a popular LUT PIOP over fields called cq [18].

PIOP, GlueLUT-v1, for a common class of integer rings where $Q = p_0 p_1 \cdots p_{s-1}$, with each p_i a large prime and $p_i < p_{i+1}$. For simplicity, we focus on the case $Q = pq$, where p and q are two distinct large primes with $p < q$.

GlueLUT-g in Figure 2 presents a general framework for constructing LUT PIOP over integer rings. A key step in GlueLUT-g is a cross-modulus consistency check, which requires range proofs to ensure that the values of the bridge object lie within $[0, Q)$. When the smallest prime factor of Q is large (for $Q = pq$ this is p), the size of the range-proof table becomes prohibitively large. While one can employ table splitting to mitigate this issue, doing so introduces additional costs from table splitting. To address this challenge, we propose a more efficient LUT PIOP specifically for $\mathbb{Z}_{pq}$, in which the cross-modulus consistency check no longer requires an additional range proof.

A natural strategy is to test consistency through a random linear combination of evaluations over the Boolean hypercube. The difficulty is that linear combinations under different moduli cannot be directly compared. We address this by choosing a sufficiently large prime P such that the linear combinations in $\mathbb{F}_P$ do not wrap around modulo P . Then computations in $\mathbb{F}_P$ coincide with integer computations, and the verifier can interpret the result as an integer and reduce it modulo Q to verify consistency modulo Q .

A standard method to form a random linear combination of multilinear evaluations over $B_\mu := \{0, 1\}^\mu$ is evaluation at a random point. For a multilinear polynomial $f(X_0, \dots, X_{\mu-1})$, choose $r = (r_0, \dots, r_{\mu-1}) \in \mathbb{F}_P^\mu$ and write

$$f(r) = \sum_{b \in B_\mu} \text{eq}(r, b) f(b), \quad \text{eq}(r, b) = \prod_{i=0}^{\mu-1} (r_i b_i + (1 - r_i)(1 - b_i)).$$

The coefficients $\text{eq}(r, b)$ contain terms of the form $1 - r_i$, which can cause wrap-around issues modulo P when we attempt to interpret the computation over $\mathbb{F}_P$ as an integer computation. To avoid this, we replace each $1 - r_i$ by an independently sampled value. A direct approach along these lines yields coefficients of size about Q^μ and forces P to grow exponentially in μ .

We reduce the required size of P by factoring the coefficient function into two parts, each depending on only $\mu/2$ variables. Write each $b \in B_\mu$ as $b = (x, y)$ with $x, y \in B_{\mu/2}$. The verifier samples coefficient tables

$$r = \{r_x\}_{x \in B_{\mu/2}} \in [0, p-1]^{B_{\mu/2}}, \quad t = \{t_y\}_{y \in B_{\mu/2}} \in [0, p-1]^{B_{\mu/2}},$$

and interprets them as the evaluation tables of multilinear polynomials $\tilde{r}, \tilde{t}$. We sample from $[0, p-1] \subset \mathbb{Z}_Q$, which forms an exceptional set, enabling a Schwartz-Zippel style bound. We define the folded sums

$$s = \sum_{x, y \in B_{\mu/2}} f(x, y) r_x t_y \in \mathbb{F}_P, \quad a = \sum_{x, y \in B_{\mu/2}} g(x, y) r_x t_y \in \mathbb{Z}_Q.$$

If all values $f(x, y)$ are canonical representatives in $[0, Q) \subset [0, P)$, then

$$0 \leq s < 2^\mu \cdot (p-1)^2 \cdot (pq-1) < 2^\mu p^3 q,$$

so choosing $P > 2^\mu p^3 q$ guarantees that s does not wrap around modulo P . In that case, the congruence test $s \equiv a \pmod{Q}$ is equivalent to integer consistency of f and g on B_μ .

The prover can prove correctness of s and a by sumcheck. Finally, we integrate the range condition on f into the lookup itself: since the lookup table values are interpreted in $[0, Q)$ and f must equal some table entry at each point, the lookup proof enforces $f(b) \in [0, Q)$ implicitly, removing the need for an additional range proof in the consistency layer. The complete PIOP is given in Figure 3.

Theorem 3. *Let $Q = pq$ with primes $p < q$, and let $n = 2^\mu$, $m = 2^\nu$, and k be the integer satisfying $(2/p)^k \leq 2^{-\lambda}$ for security parameter λ . Assume:*

- *the sumcheck protocol over a ring $\mathfrak{R}$ for μ variables and individual degree at most d has knowledge soundness error $\delta_{\mathfrak{R}, \text{sum}}^{\mu, d}$;*
- *the lookup protocol over a field $\mathbb{F}_P$ for vector length n and table size m has knowledge soundness error $\delta_{\mathbb{F}_P, \text{LUT}}^{n, m}$.*

Then the protocol in Figure 3 is a PIOP for $\mathfrak{R}_{\text{LUT}}^Q$ with:

- *Perfect completeness.*
- *Knowledge soundness error at most $\left(\frac{2}{p}\right)^k + k \cdot \delta_{\mathbb{Z}_Q, \text{sum}}^{\mu, 2} + k \cdot \delta_{\mathbb{F}_P, \text{sum}}^{\mu, 2} + \delta_{\mathbb{F}_P, \text{LUT}}^{n, m}$.*
- *PIOP complexity (excluding oracle commitment costs):*
 - *prover time: $O(n + m)$ ring/field operations;*
 - *verifier time: $O(\sqrt{n} + \log m)$ ring/field operations;*
 - *proof size: $O(\log n + \log m)$ ring/field elements.*

Proof. Completeness. Assume the prover is honest, and that the lookup table values are interpreted as canonical representatives in $[0, Q) \subset \mathbb{F}_P$. Since ϕ_i takes values in $[0, p - 1]$ and $f(b) \in [0, Q)$ for all $b \in B_\mu$, each folded value s_i is an integer strictly smaller than $2^\mu p^3 q < P$. Thus no wrap-around occurs modulo P , and $s_i \equiv a_i \pmod{Q}$ holds for all i . All sumcheck instances and the lookup proof accept, hence completeness is perfect.

Knowledge soundness. By the standard soundness-to-knowledge lifting for δ -sound PIOPs over oracle relations (cf. HyperPlonk [13] Lemma 2.3), since the witness in our protocol consists only of polynomial oracles that are already included in the instance, the δ -soundness bound established above immediately implies δ -knowledge soundness. It therefore suffices to bound the soundness error.

Suppose there exists $b^* \in B_\mu$ such that $f(b^*) \not\equiv g(b^*) \pmod{Q}$. Define

$$F_f(r, t) = \sum_{b', b'' \in B_{\mu/2}} f(b', b'') r_{b'} t_{b''}, \quad G_g(r, t) = \sum_{b', b'' \in B_{\mu/2}} g(b', b'') r_{b'} t_{b''},$$

where r and t encode the verifier sampled coefficient tables for the two halves of the hypercube. Then $F_f \not\equiv G_g \pmod{Q}$.

Consider the polynomial $F_f - G_g$ as a degree-2 polynomial in the variables r and t , and apply Lemma 1. For a random choice of (r, t) from the specified distribution,

$$\Pr[F_f(r, t) \equiv G_g(r, t) \pmod{Q}] \leq \frac{2}{p}.$$

GlueLUT-v1: Lookup Table PIOP for $\mathbb{Z}_{pq}$

- **Preprocessing.** T is a lookup table with elements from $\mathbb{Z}_Q$ with size $m = 2^\mu$ and $Q = pq$. P is a prime such that $P > 2^\mu p^3 q$. We define the lookup table polynomial $\tilde{T} \in \mathbb{F}_P[X_0, \dots, X_{\mu-1}]^{(\leq 1)}$ where for every $a \in [0, m-1]$, $\text{rep}_P(\tilde{T}(a)) = \text{rep}_Q(T[a])$.
- 1. **Commitment phase.** P commits $f \in \mathbb{F}_P[X_0, X_1, \dots, X_{\mu-1}]^{(\leq 1)}$ such that for every $b \in B_\mu$, $f(b) \equiv g(b) \pmod{Q}$ when $f(b)$ is interpreted as an integer in $[0, P)$. P sends $(P, \llbracket f \rrbracket_P)$ to V .
- 2. **Verifier sampling.** Upon receiving $(P, \llbracket f \rrbracket_P)$, V proceeds as follows:
 - (a) V samples k multilinear polynomials $\phi_0, \dots, \phi_{k-1}$ in $\mu/2 + 1$ variables. Each ϕ_i is defined by choosing its evaluations on $B_{\mu/2+1}$ independently and uniformly from $[0, p)$. The parameter k is chosen satisfying $(2/p)^k \leq 2^{-\lambda}$.
 - (b) V sends $\{\phi_i\}_{i \in [0, k-1]}$ to P .
- 3. **Sumcheck phase.** Upon receiving $\{\phi_i\}_{i \in [0, k-1]}$, P and V execute sumcheck for each $i \in [0, k-1]$:
 - (a) Sumcheck over $\mathbb{Z}_Q$:

$$a_i = \sum_{x, y \in B_{\mu/2}} g(x, y) \phi_i(0, x) \phi_i(1, y).$$

- (b) Sumcheck over $\mathbb{F}_P$:

$$s_i = \sum_{x, y \in B_{\mu/2}} f(x, y) \phi_i(0, x) \phi_i(1, y).$$

4. **Consistency check.** V checks that

$$a_i \equiv s_i \pmod{Q} \text{ for each } i \in [0, k-1].$$

5. **Lookup proof.** P and V execute a lookup proof over $\mathbb{F}_P$ to show that for every $b \in B_\mu$ there exists $b' \in B_\ell$ such that $f(b) = T(b')$ in $\mathbb{F}_P$.

Fig. 3: GlueLUT-v1: Lookup Table PIOP for $\mathbb{Z}_{pq}$

With k independent repetitions, the probability that all modulo Q consistency checks in Step 4 pass is at most $(2/p)^k$. If the sumcheck instances in Step 3 are invalid, the prover passes the check with probability at most $k \cdot \delta_{\mathbb{Z}_Q, \text{sum}}^{\mu, 2} + k \cdot \delta_{\mathbb{F}_P, \text{sum}}^{\mu, 2}$.

Finally, even if the modulo Q consistency checks hold, a prover that uses some value $f(b) \notin [0, Q)$ cannot satisfy the lookup constraint against a table whose values are interpreted in $[0, Q)$. In that case, the lookup protocol in Step 5 is accepted with probability at most $\delta_{\mathbb{F}_P, \text{LUT}}^{n, m}$.

In summary, the total knowledge soundness error is at most

$$\left(\frac{2}{p}\right)^k + k \cdot \delta_{\mathbb{Z}_Q, \text{sum}}^{\mu, 2} + k \cdot \delta_{\mathbb{F}_P, \text{sum}}^{\mu, 2} + \delta_{\mathbb{F}_P, \text{LUT}}^{n, m}.$$

PIOP complexity. There are $2k$ sumcheck instances (over $\mathbb{Z}_Q$ and over $\mathbb{F}_P$). For each instance, the prover runs in $O(n)$ ring/field operations. The verifier needs to evaluate ϕ_i at the final random point of sumcheck; since ϕ_i is given by an evaluation table over $B_{\mu/2+1}$, this costs $O(2^{\mu/2}) = O(\sqrt{n})$ ring/field operations per instance, dominated by the table size.

The lookup protocol costs $O(n + m)$ operations for the prover and $O(\log n + \log m)$ operations for the verifier⁸. The proof consists of the sumcheck transcripts (each $O(\log n)$ field elements) and the lookup transcript $O(\log n + \log m)$ field elements, hence total proof size $O(\log n + \log m)$. $\square$

Optimized based on Brakedown PCS This protocol can be further optimized when instantiated with Brakedown [25] polynomial commitments. Brakedown reshapes the evaluation vector of a multilinear polynomial on B_μ into a matrix $U \in \mathbb{F}^{2^{\mu/2} \times 2^{\mu/2}}$. Each row is encoded and committed using a Merkle tree, and the accompanying tensor IOP enables efficient linear queries to this matrix. In particular, given a verifier challenge vector $r \in \mathbb{F}^{2^{\mu/2}}$, the prover can be required to provide a vector consistent with the product $r^\top U$.

We can use this linear-query capability to streamline the consistency checks in our protocol. In the basic construction, we certify consistency between ring commitments and field commitments by repeatedly forming folded linear combinations and proving their values via sumcheck. Under Brakedown commitments, the verifier can instead query the folded values directly from the committed matrices. A full description of how this optimization is integrated into our LUT PIOP is deferred to the Appendix. [C](#).

6 GlueLUT-v2: Digit-Embedding Lookups over $\mathbb{Z}_{p^k}$

In this section, we present GlueLUT-v2: a more efficient lookup-table proof protocol over the residue ring $\mathbb{Z}_{p^k}$, which is particularly common when $p = 2$.

⁸ The complexity analysis and implementation of lookup table IOP over fields in GlueLUT-v1 are all based on the cq scheme in [18].

GlueLUT-g and GlueLUT-v1 are not directly suitable for $\mathbb{Z}_{p^k}$. The soundness analysis of GlueLUT-v1 depends on the size of an exceptional set, which is very small in $\mathbb{Z}_{p^k}$ (and becomes degenerate when $p = 2$), leading to an unacceptably large soundness error. On the other hand, GlueLUT-g can be instantiated for $\mathbb{Z}_{p^k}$, but it is suboptimal: the cross-modulus glue relies on range proofs that effectively decompose values into base- p digits, while the auxiliary prime field $\mathbb{F}_p$ is much larger than needed and thus wastes representation space.

To avoid this inefficiency, we choose the auxiliary field to be $\mathbb{F}_{p^k}$. Since $|\mathbb{F}_{p^k}| = p^k = |\mathbb{Z}_{p^k}|$, this eliminates representation waste. The remaining challenge is to enforce consistency between a witness polynomial over $\mathbb{Z}_{p^k}$ and a witness polynomial over $\mathbb{F}_{p^k}$. We use a digit-based representation map and prove consistency with respect to this map.

Definition 8 (Digit embedding and Ψ -consistency). *Let p be a prime and $k \geq 1$. Fix an $\mathbb{F}_p$ -basis $\omega = (\omega_0, \dots, \omega_{k-1})$ of $\mathbb{F}_{p^k}$. Let $\text{rep}_{p^k} : \mathbb{Z}_{p^k} \rightarrow [0, p^k)$ be the canonical representative map, and write*

$$\text{rep}_{p^k}(a) = \sum_{j=0}^{k-1} \text{dig}_j(a) p^j, \quad \text{where } \text{dig}_j(a) \in \{0, 1, \dots, p-1\}.$$

Define the digit embedding $\Psi : \mathbb{Z}_{p^k} \rightarrow \mathbb{F}_{p^k}$ by $\Psi(a) := \sum_{j=0}^{k-1} \text{dig}_j(a) \cdot \omega_j$. For functions $g : B_\mu \rightarrow \mathbb{Z}_{p^k}$ and $f : B_\mu \rightarrow \mathbb{F}_{p^k}$, define the Ψ -consistency relation

$$\text{Cons}_{p^k}^\Psi(f, g) = 1 \iff \forall b \in B_\mu, f(b) = \Psi(g(b)).$$

We use $\mathbb{F}_p$ as a bridge by checking consistency of base- p digits. Let $g \in \mathbb{Z}_{p^k}[X_0, \dots, X_{\mu-1}]^{(\leq 1)}$ be the multilinear polynomial committed in the ring-based proof. We decompose g into polynomials $g_0, \dots, g_{k-1} \in \mathbb{Z}_{p^k}[X_0, \dots, X_{\mu-1}]^{(\leq 1)}$ such that on B_μ , $g(b) = \sum_{j=0}^{k-1} p^j \cdot g_j(b)$, for all $b \in B_\mu$. We then construct $f_0, \dots, f_{k-1} \in \mathbb{F}_p[X_0, \dots, X_{\mu-1}]^{(\leq 1)}$ so that $\text{rep}_p(f_j(b)) = \text{rep}_{p^k}(g_j(b))$ for all $b \in B_\mu$ (with $g_j(b)$ interpreted as its canonical representative in $\{0, \dots, p-1\}$).

We assume that the polynomial commitment over $\mathbb{Z}_{p^k}$ supports opening at points from an extension Galois ring $\text{GR}(p^k, r)$, and that the commitment over $\mathbb{F}_p$ supports opening at points from $\mathbb{F}_{p^r}$.⁹ We will use the standard reduction map $\Phi : \text{GR}(p^k, r) \rightarrow \mathbb{F}_{p^r}$ obtained by reducing coefficients mod p to check consistency between g_j and f_j .

Fact 5 (Reduction to the residue field and a Schwartz–Zippel bound)

Let p be prime, $k, r \geq 1$, and let $\Phi : \text{GR}(p^k, r) \rightarrow \mathbb{F}_{p^r}$ be the coefficient-wise reduction modulo p as described in Fact 1. Extend Φ coordinatewise to points $\Phi : (\text{GR}(p^k, r))^\mu \rightarrow (\mathbb{F}_{p^r})^\mu$. Let $u \in \mathbb{Z}_{p^k}[X_0, \dots, X_{\mu-1}]^{(\leq 1)}$ and $v \in \mathbb{F}_p[X_0, \dots, X_{\mu-1}]^{(\leq 1)}$ be multilinear polynomials, and view v as a polynomial

⁹ Polynomial commitments over Galois rings [44,31,45] support openings over extension rings, and commitments over small fields [16,15,5] support openings over extension fields.

over $\mathbb{F}_{p^r}$ via the natural embedding $\mathbb{F}_p \hookrightarrow \mathbb{F}_{p^r}$. Sample $\tilde{\gamma} \leftarrow (GR(p^k, r))^\mu$ uniformly at random and let $\gamma := \Phi(\tilde{\gamma}) \in (\mathbb{F}_{p^r})^\mu$. If $\bar{u} \neq v$ as polynomials over $\mathbb{F}_p$, where $\bar{u}$ is obtained by reducing all coefficients of u modulo p , then

$$\Pr[\Phi(u(\tilde{\gamma})) = v(\gamma)] \leq \frac{\mu}{p^r}.$$

We now connect g over $\mathbb{Z}_{p^k}$ and f over $\mathbb{F}_{p^k}$ by: (i) proving a correct digit decomposition of g ; (ii) proving that each digit polynomial is consistent modulo p with a committed $\mathbb{F}_p$ -polynomial; (iii) proving that f is the ω -linear combination of these digit polynomials. Step (i) is enforced via a range proof and a random evaluation check over an extension ring. Step (ii) can be verified by the Fact 5 over extension rings/fields. Step (iii) is checked by a random evaluation check over an extension field. Since f_j itself is committed over $\mathbb{F}_p$, the coefficients of f must lie in $\mathbb{F}_p$, eliminating the need for additional range checks. Once $\text{Cons}_{p^k}^\Psi(f, g)$ holds, we execute a standard field lookup protocol over $\mathbb{F}_{p^k}$. The overall protocol is summarized in Figure 4.

Theorem 4. *Let $n = 2^\mu$, $m = 2^\iota$. Assume:*

- *the range proof over $\mathfrak{R}$ for vector length n and range size p has knowledge soundness error $\delta_{\mathfrak{R}, \text{range}}^{n, p}$;*
- *the lookup protocol over a field $\mathbb{F}_{p^k}$ for vector length n and table size m has knowledge soundness error $\delta_{\mathbb{F}_{p^k}, \text{LUT}}^{n, m}$.*

Then GlueLUT-v2 in Figure 4 is a PIOP for $\mathbb{Z}_{p^k}$ with:

- *Perfect completeness.*
- *Knowledge soundness error at most*

$$\frac{(k+2)\mu}{p^r} + \delta_{\mathbb{Z}_Q, \text{range}}^{kn, p} + \delta_{\mathbb{F}_{p^k}, \text{LUT}}^{n, m}.$$

- *PIOP complexity (excluding oracle commitment costs):*
 - *prover time: $O(n + m)$ ring/field operations;*
 - *verifier time: $O(\log(n + m))$ ring/field operations;*
 - *proof size: $O(\log(n + m))$ ring/field elements.*

Proof. Completeness. Completeness is straightforward to verify.

Knowledge Soundness. We invoke the same soundness-to-knowledge lifting for δ -sound PIOPs on oracle relations (cf. HyperPlonk [13] Lemma 2.3): all witnesses in GlueLUT-v2 are polynomials provided via oracle access and are part of the instance, hence the δ -soundness error proved above transfers verbatim to δ -knowledge soundness. It therefore suffices to bound the soundness error.

Assume the prover do not decompose g correctly in Step 2, there are two possible cases: (i) there exists an index $j^* \in [0, k-1]$ and a point $b^* \in B_\mu$ such that $g_{j^*}(b^*)$ is not in the range $[0, p-1]$; (ii) there exists an index $j^* \in [0, k-1]$ and a point $b^* \in B_\mu$ such that $g_{j^*}(b^*)$ is not equal to the j^* -th digit of $g(b^*)$ in base p . In case (i), the probability that the verifier accepts during the range proof in Step 3 is at most $\delta_{\mathbb{Z}_Q, \text{range}}^{kn, p}$. Assume the case (i) does not occur. In

case (ii), the multilinear polynomials $\Delta = g - \sum_{j=0}^{k-1} g_j \cdot p^j$ is non-zero. By the generalized Schwartz-Zippel lemma, the probability that the verifier accepts during the decomposition check in Step 3 is at most $\frac{\mu}{p^r}$.

So in the following, we assume that the prover has correctly decomposed g in Step 2. Assume there is $j^* \in [0, k-1]$ such that the polynomial $f_{j^*} \not\equiv g_{j^*} \pmod{p}$. By Fact 5, the probability that $f_{j^*}(\gamma) = \Phi(g_{j^*}(\tilde{\gamma}))$ in Step 4 is at most $\frac{\mu}{p^r}$. Therefore, the probability that the verifier accepts during the consistency check in Step 4 is at most $\frac{k\mu}{p^r}$. Assume that $f \neq \sum_{j=0}^{k-1} f_j \cdot \omega_j$. By the Schwartz-Zippel lemma, the probability that the verifier accepts during the consistency check in Step 4 is at most $\frac{\mu}{p^r}$.

So far, we show that if the probability that the verifier accepts in Steps 3 and 4 exceeds $\delta_{\mathbb{Z}_Q, \text{range}}^{kn,p} + \frac{\mu}{p^r} + \frac{k\mu}{p^r} + \frac{\mu}{p^r}$, then $g(x)$ is consistent with $f(x)$ in their base- p representations. Finally, if there exists a point $b^* \in B_\mu$ such that for all $b' \in B_\iota$, $f(b^*) \neq T(b')$, then the probability that the verifier accepts during the lookup proof in Step 5 is at most $\delta_{\mathbb{F}_{p^k}, \text{LUT}}^{n,m}$.

So the total knowledge soundness error is at most

$$\frac{(k+2)\mu}{p^r} + \delta_{\mathbb{Z}_Q, \text{range}}^{kn,p} + \delta_{\mathbb{F}_{p^k}, \text{LUT}}^{n,m}.$$

PIOP Complexity. Assume that the range proof over $\mathbb{Z}_Q$ and the lookup proof over $\mathbb{F}_{p^k}$ are based on memory check techniques as in Cascudo et al. [11]. In Step 3, the prover and verifier execute a range proof over $\mathbb{Z}_Q$ for k polynomials, each with vector length n and range size p . The prover complexity is $O(kn + p)$, the verifier complexity is $O(\log(kn + p))$, and the total communication is $O(\log(kn + p))$. Since k and p is a constant depending on Q and independent of n , we simplify the prover complexity to $O(n)$, the verifier complexity to $O(\log(n))$, and the total communication to $O(\log(n))$.

In Step 5, the prover and verifier execute a lookup table proof over $\mathbb{F}_{p^k}$ with vector length n and table size m . The prover complexity is $O(n + m)$, the verifier complexity is $O(\log n + \log m)$, and the communication is $O(\log n + \log m)$.¹⁰

Therefore, the overall prover complexity is $O(n + m)$, the verifier complexity is $O(\log n + \log m)$, and the total communication is $O(\log n + \log m)$. $\square$

Remark 1. Many polynomial commitment schemes over $\mathbb{F}_p$, such as Binius [16] and Binius-FRI [15], use block-level encodings that pack multiple $\mathbb{F}_p$ elements into one $\mathbb{F}_{p^k}$ element for commitment. In such instantiations, commitments to $\{f_j\}$ and to f can often be derived from the same underlying commitment, and the linear-combination check $f(\gamma) = \sum_j f_j(\gamma)\omega_j$ becomes redundant.

¹⁰ The complexity analysis and implementation of lookup table IOP over fields in GlueLUT-v2 are all based on the cq scheme in [18].

GlueLUT-v2: Lookup Table PIOP for $\mathbb{Z}_{p^k}$

- **Preprocessing Phase.** Define the table polynomial $T \in \mathbb{F}_{p^k}[X_0, \dots, X_{\ell-1}]^{(\leq 1)}$. P commits to T over $\mathbb{F}_{p^k}$ and sends the commitment to V.
- 1. **Commitment Phase.**
 - (a) P decomposes the multilinear polynomial g into base- p representation, obtaining k multilinear polynomials $g_0, g_1, \dots, g_{k-1}$, where for each $b \in B_\mu$, $g_j(b)$ represents the j -th digit of $g(b)$.
 - (b) P constructs multilinear polynomials $f_0, f_1, \dots, f_{k-1} \in \mathbb{F}_p[X_0, X_1, \dots, X_{\mu-1}]^{(\leq 1)}$ such that for each $b \in B_\mu$, $f_j(b) \equiv g_j(b) \pmod{p}$. P commits to each polynomial g_j over $\mathbb{Z}_Q$ and each polynomial f_j over $\mathbb{F}_p$.
 - (c) P constructs the multilinear polynomial $f \in \mathbb{F}_{p^k}[X_0, X_1, \dots, X_{\mu-1}]^{(\leq 1)}$, such that for each $b \in B_\mu$, $f(b) = \sum_{j=0}^{k-1} f_j(b) \cdot \omega_j$, where $\{\omega_0, \dots, \omega_{k-1}\}$ is a basis of $\mathbb{F}_{p^k}$ over $\mathbb{F}_p$. P commits to the polynomial f over $\mathbb{F}_{p^k}$.
 - (d) P sends $([[g_0]]_Q, \dots, [[g_{k-1}]]_Q, [[f_0]]_p, \dots, [[f_{k-1}]]_p, [[f]]_{\mathbb{F}_{p^k}})$ to V.
- 2. **Decomposition Check.**
 - (a) V samples a random point α from the exceptional set of extension ring $\text{GR}(p^k, r)$ where $\frac{\mu}{p^r} < 2^{-\lambda}$ and queries the oracles to get $\{(g_j(\alpha))\}_{j \in [0, k-1]}, g(\alpha)$. Then V checks that
$$g(\alpha) = \sum_{j=0}^{k-1} g_j(\alpha) \cdot p^j.$$
 - (b) P and V execute the range proof over $\mathbb{Z}_Q$ to prove that for every $b \in B_\mu$ and each $j \in [0, k-1]$, $g_j(b) \in [0, p-1]$.
- 3. **Consistency Check.** V samples a random point $\tilde{\gamma}$ from the extension ring $\text{GR}(p^k, r)$ and computes $\gamma = \Phi(\tilde{\gamma})$. Then it queries the oracles to get $\{(f_j(\gamma), g_j(\tilde{\gamma}))\}_{j \in [0, k-1]}, \{f(\gamma)\}$ (Supposed that $k \mid r$). Then V checks that
$$f(\gamma) = \sum_{j=0}^{k-1} f_j(\gamma) \cdot \omega_j, \quad \text{and} \quad f_j(\gamma) = \Phi(g_j(\tilde{\gamma})), \quad \text{for each } j \in [0, k-1].$$
- 4. **Lookup Proof.** P and V execute the LUT over $\mathbb{F}_{p^k}$ to prove that for every $b \in B_\mu$, there exists an index $b' \in B_\ell$ satisfying $f(b) = T(b')$ over $\mathbb{F}_{p^k}$.

Fig. 4: GlueLUT-v2: Lookup Table PIOP for $\mathbb{Z}_{p^k}$

7 Implementation

In this section, we provide a prototype implementation of our proposed LUT PIOPs in C++ using a single thread. All experiments were conducted on a cloud instance equipped with an Intel Xeon Platinum 8575 CPU at 2.80 GHz and 64 GB of RAM.

Our experimental evaluation aims to validate the practical efficiency of the proposed LUT PIOPs as *stand-alone* proof systems at the PIOP layer. In particular, we focus on measuring the prover/verifier’s running time and the transcript size induced by the PIOP after applying the Fiat-Shamir transform. We do not build an end-to-end application (or a full SNARK system) on top of our protocols, since doing so would require committing to a specific PCS and engineering choices that are largely orthogonal to the contributions of this work.

Implemented PIOPs and parameter choices. In this work, we implement the following two LUT PIOPs:

- GlueLUT-v1 (cf. Section 5), targeting moduli $Q = pq$ where p, q are two distinct primes. In our implementation, we fix p to be a 55-bit prime and q to be a 65-bit prime.
- GlueLUT-v2 (cf. Section 6), targeting prime-power moduli $Q = p^k$. In our implementation, we set $p := 2$ and $k := 32$, i.e. $Q = 2^{32}$.

All of our implementations achieve at least 100-bit computational security. For Fiat-Shamir transform, we use SHA256 hash function to instantiate the random oracle. Notice that, Both variants invoke a LUT PIOP over a *field* as a subroutine; we instantiate this component with a LUT PIOP called cq [18].

Performance of GlueLUT-v1. For GlueLUT-v1, to achieve at least 100-bit computational security, we set the repetition parameter $k := 3$ and we require the verifier’s challenges sampled from $\text{GR}(Q, 3)$ during the step 3a of Figure 3. we evaluate table sizes 2^4 and 2^8 and vary the input size from 2^{14} to 2^{20} . We report the prover time and the verifier time in Figure 5. Note that, We observe that changing the table size has only a minor effect on running time and has negligible impact on proof size, so we simply report the proof size here: the proof size ranges from 9.25KB to 13.44KB as the input size increases from 2^{14} to 2^{20} .

Performance of GlueLUT-v2. For GlueLUT-v2 (cf. Figure 4), profiling shows that the sumcheck protocol over Galois ring dominates prover time, verifier time and proof size across all tested input sizes. Consequently, varying the table size has little influence on the resulting performance. We therefore fix the table size to 2^8 and report the results in Figure 6.

The results in Figure 5 indicate that the prover’s running time grows approximately linearly with the input size. This behavior is consistent with the $O(n + m)$ prover complexity of GlueLUT-v1, where n denotes the input size while m denotes the table size. Likewise, the verifier’s running time follows the trend predicted by our $O(\sqrt{n} + \log m)$ verifier complexity.

We next explain the source of this bottleneck. The sumcheck subroutines used to certify the decomposition are particularly expensive when instantiated over

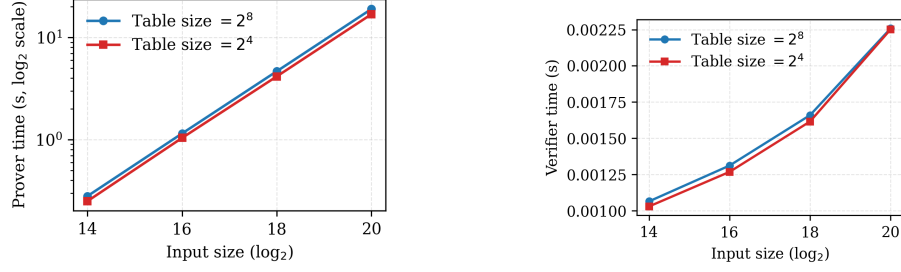


Fig. 5: Prover time and verifier time of GlueLUT-v1

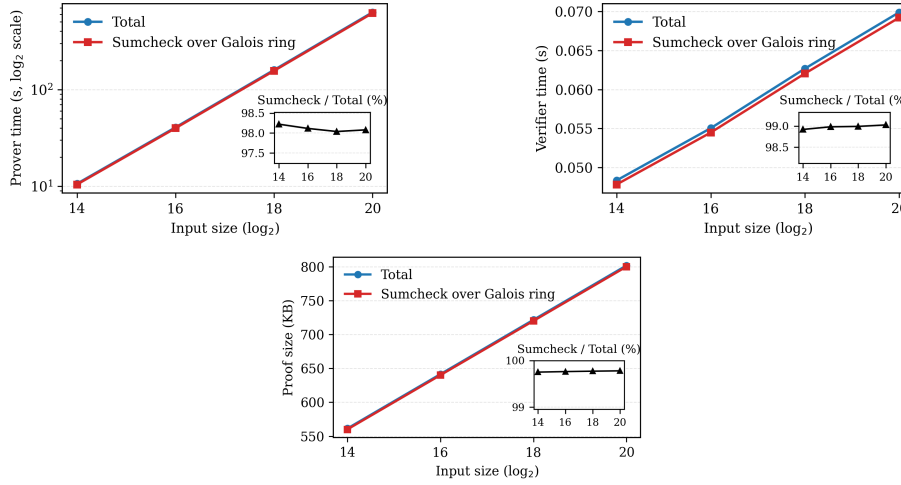


Fig. 6: Performance of GlueLUT-v2

the high-degree Galois ring needed for negligible soundness. In our setting, $Q = 2^{32}$, and the verifier's challenges are sampled from the Galois ring $\text{GR}(2^{32}, 128)$ to achieve at least 100-bit soundness.¹¹ As a result, arithmetic over $\text{GR}(2^{32}, 128)$ dominates the overall prover time in GlueLUT-v2.

Acknowledgments. We would like to thank Dario Fiore and Eduardo Soria-Vazquez for correspondence regarding their work [11].

¹¹ According to Theorem 4, when we choose $\text{GR}(2^{32}, 128)$ as the challenge space and $\mu = 20$, the knowledge soundness error of the protocol is at most $\frac{34 \cdot 20}{2^{128}} + \frac{32 \cdot 2^{20}}{2^{128}} + \frac{2^{20}}{2^{128}} < 2^{-100}$. Therefore, our implementation achieves at least 100-bit computational security.

References

1. Ambrona, M., Firsov, D., Querejeta-Azurmendi, I.: Efficient foreign-field arithmetic in PLONK. IACR Cryptol. ePrint Arch. p. 695 (2025), <https://eprint.iacr.org/2025/695>
2. Bitansky, N., Canetti, R., Chiesa, A., Tromer, E.: From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again. In: Goldwasser, S. (ed.) Innovations in Theoretical Computer Science 2012, Cambridge, MA, USA, January 8-10, 2012. pp. 326–349. ACM (2012), <https://doi.org/10.1145/2090236.2090263>
3. Bootle, J., Cerulli, A., Groth, J., Jakobsen, S.K., Maller, M.: Arya: Nearly linear-time zero-knowledge proofs for correct program execution. In: Peyrin, T., Galbraith, S.D. (eds.) Advances in Cryptology - ASIACRYPT 2018 - 24th International Conference on the Theory and Application of Cryptology and Information Security, Brisbane, QLD, Australia, December 2-6, 2018, Proceedings, Part I. Lecture Notes in Computer Science, vol. 11272, pp. 595–626. Springer (2018). https://doi.org/10.1007/978-3-030-03326-2_20, https://doi.org/10.1007/978-3-030-03326-2_20
4. Bootle, J., Chiesa, A., Sotiraki, K.: Sumcheck arguments and their applications. In: Malkin, T., Peikert, C. (eds.) Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part I. Lecture Notes in Computer Science, vol. 12825, pp. 742–773. Springer (2021), https://doi.org/10.1007/978-3-030-84242-0_26
5. Brehm, M., Chen, B., Fisch, B., Resch, N., Rothblum, R.D., Zeilberger, H.: Blaze: Fast snarks from interleaved RAA codes. In: Fehr, S., Fouque, P. (eds.) Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part IV. Lecture Notes in Computer Science, vol. 15604, pp. 123–152. Springer (2025). https://doi.org/10.1007/978-3-031-91134-7_5, https://doi.org/10.1007/978-3-031-91134-7_5
6. Bünz, B., Chen, J.: Proofs for deep thought: Accumulation for large memories and deterministic computations. In: Chung, K., Sasaki, Y. (eds.) Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part V. Lecture Notes in Computer Science, vol. 15488, pp. 269–301. Springer (2024). https://doi.org/10.1007/978-981-96-0935-2_9, https://doi.org/10.1007/978-981-96-0935-2_9
7. Bünz, B., Chen, J., DeStefano, Z.: Linear*-time permutation check. IACR Cryptol. ePrint Arch. p. 1850 (2025), <https://eprint.iacr.org/2025/1850>
8. Campanelli, M., Faonio, A., Fiore, D., Li, T., Lipmaa, H.: Lookup arguments: Improvements, extensions and applications to zero-knowledge decision trees. In: Tang, Q., Teague, V. (eds.) Public-Key Cryptography - PKC 2024 - 27th IACR International Conference on Practice and Theory of Public-Key Cryptography, Sydney, NSW, Australia, April 15-17, 2024, Proceedings, Part II. Lecture Notes in Computer Science, vol. 14602, pp. 337–369. Springer (2024). https://doi.org/10.1007/978-3-031-57722-2_11, https://doi.org/10.1007/978-3-031-57722-2_11
9. Campanelli, M., Fiore, D., Gennaro, R.: Natively compatible super-efficient lookup arguments and how to apply them. J. Cryptol. **38**(1), 14 (2025). <https://doi.org/10.1007/S00145-024-09535-0>, <https://doi.org/10.1007/s00145-024-09535-0>

10. Campanelli, M., Fiore, D., Han, S., Kim, J., Kolonelos, D., Oh, H.: Succinct zero-knowledge batch proofs for set accumulators. In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022. pp. 455–469. ACM (2022). <https://doi.org/10.1145/3548606.3560677>, <https://doi.org/10.1145/3548606.3560677>
11. Cascudo, I., Costache, A., Cozzo, D., Fiore, D., Guimarães, A., Soria-Vazquez, E.: Verifiable computation for approximate homomorphic encryption schemes. In: Kalai, Y.T., Kamara, S.F. (eds.) Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part VII. Lecture Notes in Computer Science, vol. 16006, pp. 643–677. Springer (2025). https://doi.org/10.1007/978-3-032-01907-3_21, https://doi.org/10.1007/978-3-032-01907-3_21
12. Chen, B., Tang, L., Heath, D., Kang, D.: Linear-time and logarithmically-sound permutation and multiset snarks. IACR Cryptol. ePrint Arch. p. 1967 (2025), <https://eprint.iacr.org/2025/1967>
13. Chen, B., Bünz, B., Boneh, D., Zhang, Z.: Hyperplonk: Plonk with linear-time prover and high-degree custom gates. In: Hazay, C., Stam, M. (eds.) Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part II. Lecture Notes in Computer Science, vol. 14005, pp. 499–530. Springer (2023). https://doi.org/10.1007/978-3-031-30617-4_17, https://doi.org/10.1007/978-3-031-30617-4_17
14. Chen, S., Cheon, J.H., Kim, D., Park, D.: Verifiable computing for approximate computation. IACR Cryptol. ePrint Arch. p. 762 (2019), <https://eprint.iacr.org/2019/762>
15. Diamond, B.E., Posen, J.: Polylogarithmic proofs for multilinear over binary towers. IACR Cryptol. ePrint Arch. p. 504 (2024), <https://eprint.iacr.org/2024/504>
16. Diamond, B.E., Posen, J.: Succinct arguments over towers of binary fields. In: Fehr, S., Fouque, P. (eds.) Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part IV. Lecture Notes in Computer Science, vol. 15604, pp. 93–122. Springer (2025). https://doi.org/10.1007/978-3-031-91134-7_4, https://doi.org/10.1007/978-3-031-91134-7_4
17. Dore, D.: Tassle: Lasso for the commitment-phobic. IACR Cryptol. ePrint Arch. p. 1075 (2024), <https://eprint.iacr.org/2024/1075>
18. Eagen, L., Fiore, D., Gabizon, A.: cq: Cached quotients for fast lookups. IACR Cryptol. ePrint Arch. p. 1763 (2022), <https://eprint.iacr.org/2022/1763>
19. Eagen, L., Kanjalkar, S., Ruffing, T., Nick, J.: Bulletproofs++: Next generation confidential transactions via reciprocal set membership arguments. In: Joye, M., Leander, G. (eds.) Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part V. Lecture Notes in Computer Science, vol. 14655, pp. 249–279. Springer (2024). https://doi.org/10.1007/978-3-031-58740-5_9, https://doi.org/10.1007/978-3-031-58740-5_9
20. Gabizon, A., Khovratovich, D.: flookup: Fractional decomposition-based lookups in quasi-linear time independent of table size. IACR Cryptol. ePrint Arch. p. 1447 (2022), <https://eprint.iacr.org/2022/1447>

21. Gabizon, A., Williamson, Z.J.: plookup: A simplified polynomial protocol for lookup tables. *IACR Cryptol. ePrint Arch.* p. 315 (2020), <https://eprint.iacr.org/2020/315>
22. Ganesh, C., Nitulescu, A., Soria-Vazquez, E.: Rinocchio: Snarks for ring arithmetic. *J. Cryptol.* **36**(4), 41 (2023), <https://doi.org/10.1007/s00145-023-09481-3>
23. Garreta, A., Waldner, H., Vlasov, I., Hristova, K., Dall’Ava, L., Čupić, M., Klein, M.: Zinc: Succinct arguments with small arithmetization overheads from iops of proximity to the integers. In: *Annual International Cryptology Conference*. pp. 259–291. Springer (2025)
24. Gentry, C., Wichs, D.: Separating succinct non-interactive arguments from all falsifiable assumptions. In: Fortnow, L., Vadhan, S.P. (eds.) *Proceedings of the 43rd ACM Symposium on Theory of Computing, STOC 2011, San Jose, CA, USA, 6–8 June 2011*. pp. 99–108. ACM (2011), <https://doi.org/10.1145/1993636.1993651>
25. Golovnev, A., Lee, J., Setty, S.T.V., Thaler, J., Wahby, R.S.: Brakedown: Linear-time and field-agnostic snarks for R1CS. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20–24, 2023, Proceedings, Part II. Lecture Notes in Computer Science*, vol. 14082, pp. 193–226. Springer (2023). https://doi.org/10.1007/978-3-031-38545-2_7, https://doi.org/10.1007/978-3-031-38545-2_7
26. Haböck, U.: Multivariate lookups based on logarithmic derivatives. *IACR Cryptol. ePrint Arch.* p. 1530 (2022), <https://eprint.iacr.org/2022/1530>
27. Hafezi, H., Anthoine, G., Campanelli, M., Fiore, D.: Sok: Lookup table arguments. *IACR Cryptol. ePrint Arch.* p. 1876 (2025), <https://eprint.iacr.org/2025/1876>
28. Han, S., Yoon, G., Oh, H., Kim, J.: DUPLEX: scalable zero-knowledge lookup arguments over RSA group. In: *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security, ASIA CCS 2025, Hanoi, Vietnam, August 25–29, 2025*. pp. 72–86. ACM (2025). <https://doi.org/10.1145/3708821.3733863>, <https://doi.org/10.1145/3708821.3733863>
29. Hao, M., Chen, H., Li, H., Weng, C., Zhang, Y., Yang, H., Zhang, T.: Scalable zero-knowledge proofs for non-linear functions in machine learning. *IACR Cryptol. ePrint Arch.* p. 507 (2025), <https://eprint.iacr.org/2025/507>
30. Huang, M.M., Mao, X., Zhang, J.: Sublinear proofs over polynomial rings. *IACR Cryptol. ePrint Arch.* p. 199 (2025), <https://eprint.iacr.org/2025/199>
31. Jia, Y., Li, S., Xing, C., Yao, Y., Yuan, C.: Polynomial commitments for galois rings and applications to snarks over $\mathbb{Z}_{2^k}$. In: Kalai, Y.T., Kamara, S.F. (eds.) *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17–21, 2025, Proceedings, Part VI. Lecture Notes in Computer Science*, vol. 16005, pp. 515–548. Springer (2025). https://doi.org/10.1007/978-3-032-01887-8_17, https://doi.org/10.1007/978-3-032-01887-8_17
32. Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-size commitments to polynomials and their applications. In: Abe, M. (ed.) *Advances in Cryptology - ASIACRYPT 2010 - 16th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 5–9, 2010. Proceedings. Lecture Notes in Computer Science*, vol. 6477, pp. 177–194. Springer (2010), https://doi.org/10.1007/978-3-642-17373-8_11
33. Kilian, J.: A note on efficient zero-knowledge proofs and arguments (extended abstract). In: Kosaraju, S.R., Fellows, M., Wigderson, A., Ellis, J.A. (eds.) *Proceedings of the 24th Annual ACM Symposium on Theory of Computing*, May

- 4-6, 1992, Victoria, British Columbia, Canada. pp. 723–732. ACM (1992), <https://doi.org/10.1145/129712.129782>
34. Lipton, R.J.: Efficient checking of computations. In: Choffrut, C., Lengauer, T. (eds.) STACS 90, 7th Annual Symposium on Theoretical Aspects of Computer Science, Rouen, France, February 22-24, 1990, Proceedings. Lecture Notes in Computer Science, vol. 415, pp. 207–215. Springer (1990). https://doi.org/10.1007/3-540-52282-4_44, https://doi.org/10.1007/3-540-52282-4_44
 35. Lubarov, D., Melé, J.B.: Casting out primes: Bignum arithmetic for zero-knowledge proofs. IACR Cryptol. ePrint Arch. p. 1470 (2022), <https://eprint.iacr.org/2022/1470>
 36. Lund, C., Fortnow, L., Karloff, H., Nisan, N.: Algebraic methods for interactive proof systems. Journal of the ACM (JACM) **39**(4), 859–868 (1992)
 37. Micali, S.: CS proofs (extended abstracts). In: 35th Annual Symposium on Foundations of Computer Science, Santa Fe, New Mexico, USA, 20-22 November 1994. pp. 436–453. IEEE Computer Society (1994), <https://doi.org/10.1109/SFCS.1994.365746>
 38. Papini, S., Haböck, U.: Improving logarithmic derivative lookups using GKR. IACR Cryptol. ePrint Arch. p. 1284 (2023), <https://eprint.iacr.org/2023/1284>
 39. Posen, J., Kattis, A.A.: Caulk+: Table-independent lookup arguments. IACR Cryptol. ePrint Arch. p. 957 (2022), <https://eprint.iacr.org/2022/957>
 40. Setty, S.T.V., Thaler, J.: Twist and shout: Faster memory checking arguments via one-hot addressing and increments. IACR Cryptol. ePrint Arch. p. 105 (2025), <https://eprint.iacr.org/2025/105>
 41. Setty, S.T.V., Thaler, J., Wahby, R.S.: Unlocking the lookup singularity with lasso. In: Joye, M., Leander, G. (eds.) Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part VI. Lecture Notes in Computer Science, vol. 14656, pp. 180–209. Springer (2024). https://doi.org/10.1007/978-3-031-58751-1_7, https://doi.org/10.1007/978-3-031-58751-1_7
 42. Soria-Vazquez, E.: Doubly efficient interactive proofs over infinite and non-commutative rings. In: Kiltz, E., Vaikuntanathan, V. (eds.) Theory of Cryptography - 20th International Conference, TCC 2022, Chicago, IL, USA, November 7-10, 2022, Proceedings, Part I. Lecture Notes in Computer Science, vol. 13747, pp. 497–525. Springer (2022), https://doi.org/10.1007/978-3-031-22318-1_18
 43. Wan, Z.X.: Finite fields and Galois rings. World Scientific Publishing Company (2011)
 44. Wei, Y., Zhang, X., Deng, Y.: Transparent snarks over galois rings. In: Jager, T., Pan, J. (eds.) Public-Key Cryptography - PKC 2025 - 28th IACR International Conference on Practice and Theory of Public-Key Cryptography, Røros, Norway, May 12-15, 2025, Proceedings, Part I. Lecture Notes in Computer Science, vol. 15674, pp. 418–451. Springer (2025). https://doi.org/10.1007/978-3-031-91820-9_14, https://doi.org/10.1007/978-3-031-91820-9_14
 45. Wu, Z., Zhang, X., Deng, Y., Wei, Y., Zhang, Z., Yang, L.: Polylogarithmic polynomial commitment scheme over galois rings. In: Nicomette, V., Benzekri, A., Boulahia-Cuppens, N., Vaidya, J. (eds.) Computer Security - ESORICS 2025 - 30th European Symposium on Research in Computer Security, Toulouse, France, September 22-24, 2025, Proceedings, Part II. Lecture Notes in Computer Science, vol. 16054, pp. 400–420. Springer (2025). https://doi.org/10.1007/978-3-032-07891-9_21, https://doi.org/10.1007/978-3-032-07891-9_21

- 46. Zapico, A., Buterin, V., Khovratovich, D., Maller, M., Nitulescu, A., Simkin, M.: Caulk: Lookup arguments in sublinear time. In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022. pp. 3121–3134. ACM (2022). <https://doi.org/10.1145/3548606.3560646>, <https://doi.org/10.1145/3548606.3560646>
- 47. Zapico, A., Gabizon, A., Khovratovich, D., Maller, M., Ràfols, C.: Baloo: Nearly optimal lookup arguments. IACR Cryptol. ePrint Arch. p. 1565 (2022), <https://eprint.iacr.org/2022/1565>
- 48. Zcash Developers: Lookup argument. <https://zcash.github.io/halo2/design/proving-system/lookup.html> (2025), accessed: 2025-06-23
- 49. Zhang, Y., Sun, S., Gu, D.: Efficient kzg-based univariate sum-check and lookup argument. In: Tang, Q., Teague, V. (eds.) Public-Key Cryptography - PKC 2024 - 27th IACR International Conference on Practice and Theory of Public-Key Cryptography, Sydney, NSW, Australia, April 15-17, 2024, Proceedings, Part II. Lecture Notes in Computer Science, vol. 14602, pp. 400–425. Springer (2024). https://doi.org/10.1007/978-3-031-57722-2_13, https://doi.org/10.1007/978-3-031-57722-2_13

Appendix

A Useful PIOPs over Rings [11]

Sumcheck over Rings. The sumcheck protocol [36] allows a prover to convince a verifier that the sum of evaluations of a low-degree polynomial $g : \mathfrak{R}^\mu \rightarrow \mathfrak{R}$ over the Boolean hypercube $\{0, 1\}^\mu$ is equal to the claimed value $H \in \mathfrak{R}$.

- P sends H claiming $H = \sum_{x \in \{0, 1\}^\mu} g(x)$.
- For $i = 0, \dots, \mu - 1$, P sends a univariate $g_i(X)$, intended as

$$g_i(X) = \sum_{x_{i+1}, \dots, x_{\mu-1} \in \{0, 1\}} g(r_0, \dots, r_{i-1}, X, x_{i+1}, \dots, x_{\mu-1}).$$

- V checks $\deg(g_i) \leq \deg_i(g)$ and

$$H = g_0(0) + g_0(1), \quad g_{i-1}(r_{i-1}) = g_i(0) + g_i(1) \ (i \geq 1),$$

then samples $r_i \leftarrow S \subseteq R$.

- V queries $g(r_0, \dots, r_{\mu-1})$ and accepts iff $g(r_0, \dots, r_{\mu-1}) = g_{\mu-1}(r_{\mu-1})$.

ZeroCheck over Rings. ZeroCheck [13] is a protocol that allows a prover to convince a verifier that a given low-degree polynomial $g : R^\mu \rightarrow R$ is identically zero over the Boolean hypercube $\{0, 1\}^\mu$.

- The verifier samples $\mathbf{r} \leftarrow S^\mu$ from an exceptional set $S \subset R$ and sends $\mathbf{r}$ to the prover.
- The prover and verifier execute sumcheck to prove $\sum_{\mathbf{x} \in \{0, 1\}^\mu} \text{eq}(\mathbf{r}, \mathbf{x}) \cdot g(\mathbf{x}) = 0$, where $\text{eq}(\mathbf{r}, \mathbf{x})$ is the standard multilinear equality polynomial.

B Additional Proofs

B.1 Proof of Fact 2

Fact 6 (Injectivity of the product encoding over an exceptional set) *Let $\mathfrak{R}$ be a commutative ring with identity $1 \neq 0$, and let $S \subseteq \mathfrak{R}$ be an exceptional set. For any finite multiset A over S , define the monic polynomial*

$$\Gamma_A(X) := \prod_{a \in A} (X - a) \in \mathfrak{R}[X].$$

Then the map $A \mapsto \Gamma_A(X)$ is injective on multisets supported on S : for any two finite multisets $A, B \subset S$,

$$\Gamma_A(X) = \Gamma_B(X) \text{ in } \mathfrak{R}[X] \implies A = B \text{ as multisets.}$$

Proof. We argue by induction on $n := |A|$, the total multiplicity of A .

If $n = 0$, then $\Gamma_A(X) = 1$, hence $\Gamma_B(X) = 1$ and B must be empty as well. Thus $A = B$.

Assume $n \geq 1$ and pick an element $a \in S$ with positive multiplicity in A . Then $\Gamma_A(a) = 0$ since $(X - a)$ divides $\Gamma_A(X)$, and hence also $\Gamma_B(a) = 0$ because $\Gamma_A = \Gamma_B$.

Write $B = \{b_1, \dots, b_n\}$ with multiplicities, so

$$\Gamma_B(a) = \prod_{i=1}^n (a - b_i).$$

If a does not occur in B , then for every i we have $a \neq b_i$ and, since both lie in the same exceptional set S , the difference $a - b_i$ is a unit in $\mathfrak{R}$. Therefore $\Gamma_B(a)$ is a product of units and hence itself a unit; in particular $\Gamma_B(a) \neq 0$. This contradicts $\Gamma_B(a) = 0$. Hence a must occur in B .

Remove one occurrence of a from each multiset and denote the resulting multisets by $A' := A \setminus \{a\}$ and $B' := B \setminus \{a\}$. Then

$$\Gamma_A(X) = (X - a) \Gamma_{A'}(X) \quad \text{and} \quad \Gamma_B(X) = (X - a) \Gamma_{B'}(X).$$

For monic divisor $X - a$, the usual long division in $\mathfrak{R}[X]$ results in a unique quotient and remainder. Thus from $\Gamma_A(X) = \Gamma_B(X)$ it follows that $\Gamma_{A'}(X) = \Gamma_{B'}(X)$.

Now $|A'| = n - 1$, so by the induction hypothesis we get $A' = B'$ as multisets, and therefore $A = B$. $\square$

B.2 Proof of Lemma 2

Lemma 4 (Lemma 2). *Let p and q be prime numbers. Then there exist distinct subsets $A, B \subseteq \mathbb{Z}_{pq}$ with $A \neq B$ such that $\prod_{a \in A} (x - a) = \prod_{b \in B} (x - b)$ as polynomials over $\mathbb{Z}_{pq}$.*

Proof. Consider a subset $A = \{a_0, \dots, a_{n-1}\} \subseteq \mathbb{Z}_{pq}$. For each i , define $a_{p,i} = a_i \bmod p$ and $a_{q,i} = a_i \bmod q$, and let

$$\Gamma_A(x) = \prod_{i=0}^{n-1} (x - a_i) \in \mathbb{Z}_{pq}[x],$$

so that

$$\Gamma_A(x) = c_0 + c_1 x + \dots + c_n x^n \pmod{pq}.$$

By the Chinese Remainder Theorem, $\Gamma_A(x)$ is determined by its reductions modulo p and q , namely

$$\Gamma_p(x) = c_{p,0} + c_{p,1}x + \dots + c_{p,n}x^n \in \mathbb{Z}_p[x], \quad \Gamma_q(x) = c_{q,0} + c_{q,1}x + \dots + c_{q,n}x^n \in \mathbb{Z}_q[x],$$

where $c_{p,i} = c_i \bmod p$ and $c_{q,i} = c_i \bmod q$. The multiset $\{a_{p,i}\}$ consists of the roots of $F_p(x)$ in $\mathbb{Z}_p$, and we write $A_p = \{a_{p,0}, \dots, a_{p,n-1}\} \subseteq \mathbb{Z}_p$. Similarly,

$\{a_{q,i}\}$ is the multiset of roots of $F_q(x)$ in $\mathbb{Z}_q$, and $A_q = \{a_{q,0}, \dots, a_{q,n-1}\} \subseteq \mathbb{Z}_q$. Since these roots are unordered, for any permutation σ of $\{0, \dots, n-1\}$, the paired multiset $\{(a_{p,i}, a_{q,\sigma(i)})\}_i$ still corresponds to the same pair of polynomials (F_p, F_q) . By the Chinese Remainder isomorphism, each such pairing yields a subset $A^\sigma = \{a_0^\sigma, \dots, a_{n-1}^\sigma\} \subseteq \mathbb{Z}_{pq}$ whose associated product $\prod_i (x - a_i^\sigma)$ is equal to $F(x)$ in $\mathbb{Z}_{pq}[x]$. Thus, different subsets A^σ of $\mathbb{Z}_{pq}$ can give rise to the same polynomial $F_A(x)$. $\square$

B.3 Proof of Lemma 3

Lemma 5 (Lemma 3). *Let p be a prime and k a positive integer. For $p \geq 5$ and $k \geq 2$, or for $p \in \{2, 3\}$ and $k \geq 3$, there exist distinct subsets $A, B \subseteq \mathbb{Z}_{p^k}$ with $A \neq B$ such that $\prod_{a \in A} (x - a) = \prod_{b \in B} (x - b)$ as polynomials over $\mathbb{Z}_{p^k}$.*

Proof. For the case $k \geq 3$, the subsets $\{0, p^{k-1} + p\}$ and $\{p, p^{k-1}\}$ of $\mathbb{Z}_{p^k}$ are indistinguishable in the sense that they define the same polynomial over $\mathbb{Z}_{p^k}$. Indeed, in $\mathbb{Z}_{p^k}[x]$ we have

$$(x-p)(x-p^{k-1}) = x^2 - (p^{k-1}+p)x + p^k \equiv x^2 - (p^{k-1}+p)x = x(x - (p^{k-1}+p)) \pmod{p^k},$$

so $\{0, p^{k-1} + p\}$ and $\{p, p^{k-1}\}$ cause the same product polynomial. Consequently, for any subset $C \subseteq \mathbb{Z}_{p^k}$, the unions $C \cup \{0, p^{k-1} + p\}$ and $C \cup \{p, p^{k-1}\}$ are also indistinguishable in this sense.

For the case $k = 2$ and $p \geq 5$, the subsets $\{1, 1 + p\}$ and $\{1 + 2p, 1 - p\}$ of $\mathbb{Z}_{p^2}$ are likewise indistinguishable. Indeed,

$$(x-1)(x-(1+p)) = x^2 - (p+2)x + (p+1), \pmod{p^2}$$

and

$$(x-(1+2p))(x-(1-p)) = x^2 - (p+2)x + (1+p-2p^2) \equiv x^2 - (p+2)x + (p+1) \pmod{p^2}.$$

Thus both pairs generate the same polynomial over $\mathbb{Z}_{p^2}$. As before, for any subset $C \subseteq \mathbb{Z}_{p^2}$, the unions $C \cup \{1, 1 + p\}$ and $C \cup \{1 + 2p, 1 - p\}$ are indistinguishable with respect to the induced product polynomial. $\square$

B.4 Proof of Fact 3

Fact 7 (Fact 3) *Let P, Q be two coprime integers and $P > Q$. Let $x \in [0, P)$, $y \in [0, Q)$, and $z \in [0, PQ)$. If*

$$z \equiv x \pmod{P}, \quad z \equiv y \pmod{Q}, \quad \text{and} \quad z \in [0, Q),$$

then $x = z$ and $y = z$ as integers. In particular, $x = y$ as integers in $[0, Q)$.

Proof. We prove the fact by contradiction. Assume for contradiction that $x \neq z$. Because $x \in [0, P-1]$ and $z \in [0, Q-1] \subseteq [0, P-1]$ (as $P > Q$) we have $0 < |x - z| < P$. However, $z \equiv x \pmod{P}$ implies $P \mid (x - z)$, contradicting $0 < |x - z| < P$. Hence $x = z$.

It remains to show that $y = z$. Assume for contradiction that $y \neq z$. Because $y \in [0, Q-1]$ and $z \in [0, Q-1]$, we have $0 < |y - z| < Q$. On the other hand, $z \equiv y \pmod{Q}$ implies $Q \mid (y - z)$, which is contradictory to $0 < |y - z| < Q$. Therefore $y = z$. Together with $x = z$, we conclude $x = y = z$. $\square$

B.5 Proof of Fact 4

Fact 8 (Fact 4) *Let $P, Q \geq 2$ be coprime integers, let $N = PQ$, and let s be the smallest prime divisor of N . Let $\ell = \lceil \log_s Q \rceil$, and write the base- s expansion*

$$Q - 1 = \sum_{i=0}^{\ell-1} q'_i s^i, \quad q'_i \in \{0, 1, \dots, s-1\}.$$

Let $h \in \mathbb{Z}_N$. Suppose there exist integers $h_0, \dots, h_{\ell-1}$, $a_0, \dots, a_{\ell-1}$, and $\beta_0, \dots, \beta_{\ell}$ in $\{0, 1, \dots, N-1\}$ such that:

1. $h \equiv \sum_{i=0}^{\ell-1} h_i s^i \pmod{N}$ with $h_i \in \{0, 1, \dots, s-1\}$ for all i ;
2. $a_i \in \{0, 1, \dots, s-1\}$ for all i , and $\beta_i \in \{0, 1\}$ for all i , with $\beta_0 = \beta_{\ell} = 0$;
3. for every $i = 0, \dots, \ell-1$: $h_i + \beta_i + a_i \equiv q'_i + s \cdot \beta_{i+1} \pmod{N}$.

Then $h \in [0, Q-1]$ as an integer.

Proof. Fix $i \in \{0, \dots, \ell-1\}$. Since $h_i, a_i \in \{0, \dots, s-1\}$ and $\beta_i, \beta_{i+1} \in \{0, 1\}$, both integers $h_i + \beta_i + a_i$ and $q'_i + s \beta_{i+1}$ belong to $[0, 2s-1]$. Hence their difference $\Delta_i = (h_i + \beta_i + a_i) - (q'_i + s \beta_{i+1})$ satisfies $|\Delta_i| \leq 2s-1$. Because $\Delta_i \equiv 0 \pmod{N}$ by assumption and $N = PQ \geq s^2 > 2s-1$, it follows that $|\Delta_i| < N$ and therefore $\Delta_i = 0$. Consequently,

$$h_i + \beta_i + a_i = q'_i + s \beta_{i+1}$$

holds as an equality in $\mathbb{Z}$ for every i .

Let $H = \sum_{i=0}^{\ell-1} h_i s^i$. Since each $h_i \in [0, s-1]$, one has $0 \leq H \leq s^{\ell} - 1$, hence $H < s^{\ell}$. By definition of $\ell = \lceil \log_s Q \rceil$, we have $Q \leq s^{\ell}$, so $H < s^{\ell} \leq sQ$. Moreover, since s is the smallest prime divisor of N , we have $P \geq s$, hence $N = PQ \geq sQ$. Therefore $H < N$, and since also $H \geq 0$, we have $H \in \{0, 1, \dots, N-1\}$. The congruence $h \equiv H \pmod{N}$ then implies that the canonical representative of h in $\{0, \dots, N-1\}$ is exactly H . Now multiply the equalities $h_i + \beta_i + a_i = q'_i + s \beta_{i+1}$ by s^i and sum over $i = 0, \dots, \ell-1$ to obtain

$$\sum_{i=0}^{\ell-1} (h_i + \beta_i + a_i) s^i = \sum_{i=0}^{\ell-1} q'_i s^i + \sum_{i=0}^{\ell-1} \beta_{i+1} s^{i+1}.$$

The middle sum equals $Q - 1$ by the base- s expansion. Using $\beta_0 = \beta_\ell = 0$, we also have

$$\sum_{i=0}^{\ell-1} \beta_i s^i = \sum_{i=1}^{\ell-1} \beta_i s^i \quad \text{and} \quad \sum_{i=0}^{\ell-1} \beta_{i+1} s^{i+1} = \sum_{i=1}^{\ell} \beta_i s^i = \sum_{i=1}^{\ell-1} \beta_i s^i.$$

Therefore

$$\sum_{i=0}^{\ell-1} h_i s^i + \sum_{i=0}^{\ell-1} a_i s^i = Q - 1.$$

Let $A = \sum_{i=0}^{\ell-1} a_i s^i$. Then $A \geq 0$, and the previous display gives $H + A = Q - 1$. Since the canonical representative of h equals H , we conclude that $Q - 1 - h = A \geq 0$, so $h \leq Q - 1$. Also $h \geq 0$ because $h = H \geq 0$. Therefore h lies in $[0, Q - 1]$, as claimed. $\square$

C Optimized based on Brakedown PCS

In this section, we show how to optimize GlueLUT-v1 when the underlying PCS is instantiated with Brakedown, which is widely-used. Before presenting our formal protocol, we will provide a quick recap of Brakedown.

C.1 A Quick Recap of Brakedown PCS

Given a function $f : \{0, 1\}^\mu \rightarrow \mathfrak{R}$, its *multilinear extension* (MLE) is the (unique) multilinear polynomial $\tilde{f} : \mathfrak{R}^\mu \rightarrow \mathfrak{R}$ such that $f(b) = \tilde{f}(b)$ for all $b \in \{0, 1\}^\mu$. The MLE of f is the following polynomial

$$\tilde{f}(X_1, \dots, X_\mu) = \sum_{b \in \{0, 1\}^\mu} f(b) \cdot \chi_b(X_1, \dots, X_\mu),$$

where

$$\chi_b(X_1, \dots, X_\mu) = \prod_{k=1}^{\mu} \chi_{b_k}(X_k), \quad \text{with } \chi_1(X) = X \text{ and } \chi_0(X) = 1 - X.$$

Brakedown Polynomial Commitment over Rings [44, 31, 11]. Let $\mathfrak{R}$ be ring and let $f : \{0, 1\}^\mu \rightarrow \mathfrak{R}$ be a multilinear polynomial. Write $n = 2^\mu$ for the evaluation length. For ease of exposition, assume $n = m^2$ and reshape the evaluation vector of f into an $m \times m$ matrix $U \in \mathfrak{R}^{m \times m}$, whose i -th row is $U_i \in \mathfrak{R}^m$.

Brakedown fixes a linear error-correcting code $\text{Enc} : \mathfrak{R}^m \rightarrow \mathfrak{R}^{\gamma m}$ with constant rate and constant relative distance, and a Merkle commitment scheme for vectors over $\mathfrak{R}$.

- **Commit Phase.** The committer encodes each row independently:

$$\widehat{U}_i := \text{Enc}(U_i) \in \mathfrak{R}^{\gamma^m}, \quad i \in [m],$$

obtaining the encoded matrix $\widehat{U} \in \mathfrak{R}^{m \times \gamma^m}$. It then commits to the entire $\widehat{U}$ via a Merkle tree, and outputs the Merkle root com .

- **Testing Phase.** The verifier samples a length- m random challenge vector α from the exceptional set of $\mathfrak{R}$. The prover forms the row-wise random linear combination

$$u^{(0)} := \sum_{i=1}^m \alpha_i \cdot U_i \in \mathfrak{R}^m$$

and sends $u^{(0)}$ to the verifier. The verifier chooses a random query set $E \subseteq [n]$ of size $|E| = \Theta(\lambda)$. For each queried column index $j \in E$, the prover opens the entire column $(\widehat{U}_{1,j}, \dots, \widehat{U}_{m,j})$ together with the corresponding Merkle authentication paths. The verifier checks the Merkle openings against com and verifies the linear constraint

$$\text{Enc}(u^{(0)})_j \stackrel{?}{=} \sum_{i=1}^m \alpha_i \cdot \widehat{U}_{i,j} \quad \text{for all } j \in E.$$

- **Evaluation Phase.** To open f at a point $\mathfrak{R} \in \mathbb{F}^\mu$ the verifier deterministically derives (from r) two vectors $q_1, q_2 \in \mathfrak{R}^m$ such that the claimed value satisfies

$$g(r) = \langle q_1 \otimes q_2, U \rangle,$$

where $\otimes$ denotes the tensor product and $\langle \cdot, \cdot \rangle$ is the standard inner product. The prover sends the partial contraction

$$u^{(1)} := \sum_{i=1}^m (q_1)_i \cdot U_i \in \mathfrak{R}^m.$$

The verifier repeats the same spot-checking procedure as in the testing phase with challenge vector q_1 : it samples a fresh query set $E' \subseteq [n]$ of size $\Theta(\lambda)$, requests openings of the corresponding columns of $\widehat{U}$, and checks

$$\text{Enc}(u^{(1)})_j \stackrel{?}{=} \sum_{i=1}^m (q_1)_i \cdot \widehat{U}_{i,j} \quad \text{for all } j \in E'.$$

If all checks pass, the verifier outputs the opened value $v := \langle u^{(1)}, q_2 \rangle$, which equals $f(r)$.

C.2 Optimized LUT Argument

Our optimized protocol is put in Figure 7. We prove the security of this protocol through the following theorem.

Optimized LUT argument for $\mathbb{Z}_{pq}$ based on Brakedown PCS

- **Preprocessing.** T is a lookup table with elements from $\mathbb{Z}_Q$ with size $m = 2^\ell$ and $Q = pq$. P is a prime such that $P > 2^\mu p^2 q$. We define the lookup table polynomial $\tilde{T} \in \mathbb{F}_P[X_0, \dots, X_{\ell-1}]^{(\leq 1)}$ where for every $a \in [0, m-1]$, $\text{rep}_P(\tilde{T}(a)) = \text{rep}_Q(T[a])$.
- 1. **Commitment Phase.** P commits $f \in \mathbb{F}_P[X_0, \dots, X_{\mu-1}]^{(\leq 1)}$ such that for all $b \in B_\mu$, $\text{rep}_P(f(b)) = \text{rep}_Q(g(b))$ using Brakedown over $\mathbb{F}_P$, resulting in commitment $[[f]]_P$.
- 2. **Verifier Sampling.** Upon receiving $([[f]]_P)$, the verifier proceeds as follows.
 - (a) It samples k ($\frac{1}{p^k} < 2^{-\lambda}$) random vectors $r_0, \dots, r_{k-1}$ from $[0, p-1]^{2^{\mu/2}}$.
 - (b) It sends $\{r_i\}_{i \in [k]}$ to P .
- 3. **Prover Responses.** Suppose the coefficient matrices of f and g are $U_f \in \mathbb{F}_P^{2^{\mu/2} \times 2^{\mu/2}}$ and $U_g \in \mathbb{Z}_Q^{2^{\mu/2} \times 2^{\mu/2}}$, respectively. Upon receiving $\{r_i\}_{i \in [k]}$, the prover computes:
 - (a) For each $i \in [k]$, it computes $v_i := r_i^\top U_f$, in $\mathbb{F}_P^{2^{\mu/2}}$ together with a Brakedown opening proof π_i^P attesting that $v(i)$ is the correct response of $[[f]]_P$ to query r_i . It also computes $u_i := r_i^\top U_g$, in $\mathbb{Z}_Q^{2^{\mu/2}}$ together with an opening proof π_i^Q for $[[g]]_Q$ on the same query r_i .
 - (b) It sends $\{(v_i, \pi_i^P, u_i, \pi_i^Q)\}_{i \in [k]}$ to V .
- 4. **Verification.** Upon receiving the prover's message, the verifier:
 - (a) verifies all opening proofs $\{(v_i, \pi_i^P, u_i, \pi_i^Q)\}_{i \in [k]}$ with respect to the corresponding commitments and queries;
 - (b) for all $i \in [k]$ and all coordinates $j \in [2^{\mu/2}]$, checks $v_{i,j} \equiv u_{i,j} \pmod{Q}$;
- 5. **Lookup Proof.** P and V execute a lookup proof over $\mathbb{F}_P$ to show that for every $b \in B_\mu$ there exists $b' \in B_\ell$ such that $f(b) = \tilde{T}(b')$ in $\mathbb{F}_P$.

Fig. 7: Optimized LUT argument for $\mathbb{Z}_{pq}$ based on Brakedown PCS

Theorem 5. Let $n = 2^\mu$. Assume that over a ring $\mathcal{R}$, the Brakedown protocol for committing to a μ -variate multilinear polynomial has knowledge soundness error $\delta_{\mathcal{R}, \text{Brakedown}}^\mu$, and that a lookup protocol over $\mathbb{F}$ with vector length n and table size T has knowledge soundness error $\delta_{\mathbb{F}, \text{LUT}}^{n, T}$. Given the multilinear $g \in \mathbb{F}_q[X_0, \dots, X_{\mu-1}]^{(\leq 1)}$, Protocol 7 satisfies the following properties:

- Completeness holds perfectly.
- The prover complexity is $O(n + m)$ ring operations.
- The verifier complexity is $O(\sqrt{n} + \log(n + m))$ ring operations.
- The total number of field elements sent by the prover is $O(\sqrt{n} + \log(n + m))$.

Suppose that the prover passes

- the lookup verification in Step 5 with probability greater than $\delta_{\mathbb{F}_P, \text{LUT}}^{n, m}$,
- the modular consistency checks in Step 4 with probability greater than $\frac{1}{p^k}$,
- and the verification of correct linear combinations in Step 4 with probability greater than $\delta_{\mathbb{F}_P, \text{Brakedown}}^\mu + \delta_{\mathbb{F}_Q, \text{Brakedown}}^\mu$.

Then it holds that $\forall b \in B_\mu$, $\text{rep}_P(f(b)) = \text{rep}_Q(g(b))$.

Proof. Completeness. The completeness of the protocol is straightforward.

One point worth noting is that we require $P > 2^\mu p^2 q$ instead of $P > 2^\mu p^3 q$. This is because, in this optimized protocol, the linear combination challenge consists of a single element rather than the product of two elements. The value of this single element is chosen from the range $[0, p - 1]$. Therefore, to ensure that the result of the linear combination does not exceed $2^\mu pq$, it suffices to have $P > 2^\mu p^2 q$.

Soundness. Assume that the prover passes: (i) the lookup verification in Step 5 with probability $> \delta_{\mathbb{F}_P, \text{LUT}}^{n, m}$, (ii) the modular consistency checks in Step 4 with probability $> 1/p^k$, and (iii) the verification of the Brakedown openings with probability $> \delta_{\mathbb{F}_P, \text{Brakedown}}^\mu + \delta_{\mathbb{Z}_Q, \text{Brakedown}}^\mu$. We show that then necessarily $\forall b \in B_\mu$, $\text{rep}_P(f(b)) = \text{rep}_Q(g(b))$, equivalently $f(b) = g(b)$ under the canonical representatives.

By assumption (iii) and knowledge soundness of Brakedown, except with probability at most $\delta_{\mathbb{F}_P, \text{Brakedown}}^\mu + \delta_{\mathbb{Z}_Q, \text{Brakedown}}^\mu$, all accepted opening proofs in Step 4 imply that for every $i \in [k]$,

$$v_i = r_i^\top U_f \in \mathbb{F}_P^{2^{\mu/2}} \quad \text{and} \quad u_i = r_i^\top U_g \in \mathbb{Z}_Q^{2^{\mu/2}},$$

where U_f, U_g are the matrix forms of the evaluation tables of f and g on B_μ .

Assume for contradiction that f and g are *not* pointwise consistent as integers. Then there exists a point $(x^*, y^*) \in B_{\mu/2} \times B_{\mu/2}$ such that

$$\text{rep}_P(f(x^*, y^*)) \neq \text{rep}_Q(g(x^*, y^*)).$$

Since rep_P is the canonical representative in $[0, P)$ and $P > 2^\mu p^2 q \geq pq = Q$, this implies in particular that the residues disagree modulo Q :

$$f(x^*, y^*) \not\equiv g(x^*, y^*) \pmod{Q}.$$

Fix the column y^* and define the difference vector over $\mathbb{Z}_Q$:

$$\Delta_{y^*} \in \mathbb{Z}_Q^{2^{\mu/2}}, \quad (\Delta_{y^*})_x := f(x, y^*) - g(x, y^*) \pmod{Q}.$$

By assumption, $\Delta_{y^*} \neq 0$ in $\mathbb{Z}_Q^{2^{\mu/2}}$. Consider one verifier query $r \leftarrow [0, p-1]^{2^{\mu/2}}$ and the folded difference $D(r) := r^\top \Delta_{y^*} \in \mathbb{Z}_Q$. We claim that

$$\Pr_{r \leftarrow [0, p-1]^{2^{\mu/2}}} [D(r) \equiv 0 \pmod{Q}] \leq \frac{1}{p}.$$

To see this, reduce modulo p . Let $\overline{\Delta}_{y^*}$ be $\Delta_{y^*} \bmod p$ in $\mathbb{F}_p^{2^{\mu/2}}$. Since $\Delta_{y^*} \neq 0$ in $\mathbb{Z}_Q$, we also have $\overline{\Delta}_{y^*} \neq 0$ in $\mathbb{F}_p$ for at least one coordinate. Pick an index x_0 such that $(\overline{\Delta}_{y^*})_{x_0} \neq 0$. Condition on all coordinates of r except r_{x_0} . Then the equation $D(r) \equiv 0 \pmod{p}$ becomes a non-degenerate linear equation in the single uniform variable $r_{x_0} \in \mathbb{F}_p$, hence it has exactly one solution. Therefore

$$\Pr[D(r) \equiv 0 \pmod{p}] = \frac{1}{p},$$

and since $D(r) \equiv 0 \pmod{Q}$ implies $D(r) \equiv 0 \pmod{p}$, the claimed bound follows. Because the r_i are sampled independently,

$$\Pr[\forall i \in [k], D(r_i) \equiv 0 \pmod{Q}] \leq \left(\frac{1}{p}\right)^k.$$

Now observe that, for the particular column y^* , the verifier's coordinate-wise checks $v_{i, y^*} \equiv u_{i, y^*} \pmod{Q}$ in Step 4 are exactly testing $D(r_i) \equiv 0 \pmod{Q}$. Hence, if f and g are inconsistent at some point, the probability that all k modular checks pass is at most $1/p^k$, contradicting assumption (ii). So we conclude that except with probability at most

$$\frac{1}{p^k} + \delta_{\mathbb{F}_P, \text{Brakedown}}^\mu + \delta_{\mathbb{Z}_Q, \text{Brakedown}}^\mu,$$

the acceptance of Step 4 implies that no inconsistency point (x^*, y^*) exists. Thus, with the stated probability, for all $(x, y) \in B_{\mu/2} \times B_{\mu/2}$ we have $f(x, y) \equiv g(x, y) \pmod{Q}$. Since Step 5 ensures (except with probability $\delta_{\mathbb{F}_P, \text{LUT}}^{n, m}$) that $f(B_\mu) \subseteq \tilde{T}(B_\mu)$ and $\tilde{T}$ is defined using canonical representatives (Preprocessing), all $f(b)$ lie in the intended integer range and therefore $\text{rep}_P(f(b)) = \text{rep}_Q(g(b))$ holds for all $b \in B_\mu$ as claimed.

Complexity. *Prover.* In Step 3 the prover answers k Brakedown linear queries for $[[f]]_P$ and $[[g]]_Q$ and produces the corresponding opening proofs. Brakedown supports such linear queries with $O(2^{\mu/2}) = O(\sqrt{n})$ work per query, hence $O(k\sqrt{n})$ total, and $k = O(\lambda/\log p)$ is independent of n . In Step 5 the prover runs a lookup argument over $\mathbb{F}_P$ for vector length n and table size m , costing $O(n+m)$. Overall, the prover complexity is $O(n+m)$.

Verifier. In Step 4, verifying k Brakedown opening proofs costs $O(k\sqrt{n}) = O(\sqrt{n})$, and the verifier performs a coordinate-wise modular check on k vectors of length $2^{\mu/2}$, which costs $O(k\sqrt{n}) = O(\sqrt{n})$ ring operations. Step 5 costs $O(\log(n+m))$ for a standard lookup protocol verifier. Overall, the verifier complexity is $O(\sqrt{n} + \log(n+m))$.

Communication. Step 3 sends k vectors v_i, u_i of length $2^{\mu/2}$, plus k opening proofs for $[[f]]_P$ and $[[g]]_Q$, each of size $O(\sqrt{n})$. Step 5 contributes $O(\log(n+m))$ field elements. Hence total communication is $O(\sqrt{n} + \log(n+m))$ field/ring elements. $\square$